NATIONAL UNIVERSITY OF LESOTHO

Department of Mathematics and Computer Science

CS4433 Software Engineering: Final Mini Project

STUDENT NUMBER: 202321162

STUDENT NAME: LEHLOHONOLO

STUDENT SURNAME: MAJORO

COURSE CODE: CS 4433

PROJECT: LEARNING MANAGEMENT SYSTEM

PHASE 3: Architecture, Class Design, Behavioural Models and Database Design

1. Introduction

1.1 Purpose and scope of this document

Phase 2 defined what the Moderated Learning Management System (LMS) must do: 35 functional requirements, 15 non-functional requirements and 15 business rules. This Phase 3 document defines how the system will be built, so that the implementation (Phase 4) can be checked against a design that has passed Gate 2. Under the project rules, implementation may not start until Gate 2 is passed.

Database product. The design uses My SQL, as required by the project brief and as already stated in SRS Section 4.3.1. The words “My SQL” in Phase 1 and in SRS Section 4.1.2 are corrected to SQLite through the change-control procedure of the project plan.

1.2 Method used

The design was produced in four steps:

- **Drive.** Architectural drivers were taken from the Must non-functional requirements, the constraints, console, offline, one developer, about 210 hours and the business rules BR-01 to BR-15.
- **Choose.** Every major decision (architectural style, patterns, handling of the submission life cycle, storage of marks) was compared with at least one alternative. The choice and the reason are written down in the decision register (Table 1.1).
- **Model.** The chosen design was drawn in UML (component, class, state machine, activity and sequence diagrams) and as an entity-relationship diagram (ERD), using the same class, status and table names in every diagram.
- **Check.** Every functional requirement was traced to classes, tables and diagrams (Section 6). The diagrams were then compared with each other for matching names, statuses and rules, and the result is recorded in the Gate 2 checklist (Section 7).

1.3 Key design decisions

Table 1.1 lists the decisions that shape the rest of the document. Each one has an identifier (DD-xx) so that later phases can refer to it.

Table 1.1: Design decision register

ID	Decision	Main reason
DD-01	Layered architecture: presentation, business logic, data access, plus infrastructure services.	Fits a console program; keeps SQL out of the menus (NFR-06); allows unit tests of the business layer (NFR-12).

ID	Decision	Main reason
DD-02	Late-penalty rule isolated behind an IPenaltyStrategy interface (Strategy pattern).	BR-05 to BR-07 are the rules most likely to be disputed (CF2, CF3) and most error-prone (R13); one class to test.
DD-03	All database access through Repository interfaces implemented with SQLite.	Required by the brief; parameterised queries in one place (NFR-06); test doubles possible.
DD-04	Abstract User class with four subclasses created by a UserFactory.	Required by the brief; permitted functions differ by role and the menu is built from them (FR-01, FR-03).
DD-05	Menu actions are Command objects; menus are generated from the user's permitted functions.	A user can only see what the role allows; at most 3 menu levels (NFR-10).
DD-06	Submission life cycle stored as a transition table (SubmissionLifecycle), not as six State classes.	The six states differ only in which transitions are allowed, so a table is simpler and fully testable (FR-28).
DD-07	Every mark change, its status change and its audit record are written inside one RAII Transaction.	NFR-09 and FR-29 require that no mark changes without an audit record.
DD-08	All times are held and stored in UTC (ISO-8601 text) and shown in local time.	Deadlines and penalties depend on time (R13); avoids time-zone and daylight-saving errors.
DD-09	Marks are kept in their own table, one row per marked submission.	A submission that is not yet marked carries no empty mark columns; mark data has its own life.
DD-10	One users table with a role column (single-table inheritance).	Four roles share almost all columns; avoids four tables and many joins.
DD-11	Audit records are append-only, enforced by the application and by database triggers.	Accountability required by the Data Protection Act and FR-29.

2. Architecture

2.1 Architectural drivers

The requirements that most affect the structure of the system are listed in Table 2.1. Functional requirements describe features; these drivers decide how the features are arranged.

Table 2.1: Architectural drivers and their effect

Driver	Source	Effect on the architecture
Works offline on old computers; one school computer or local network	NFR-03, NFR-04, SRS 4.2.4	Single process, embedded SQLite database, no network libraries, console interface, small memory footprint.
Login within 2 s and operations within 1 s with 800 students and 20,000 submissions	NFR-01, NFR-02, CF7	Indexed queries (Section 5.7); only one password hash per login; lists fetch only the rows needed.
Passwords hashed, SQL parameterised, roles enforced	NFR-05, NFR-06, NFR-07, FR-03	IPasswordHasher (Argon2id); one data-access layer with prepared statements; PermissionPolicy called in every service method and menus built from the role.
A mark and its audit record must never be separated	NFR-09, FR-29, BR-12	RAII Transaction around every mark change; audit written by the same transaction; append-only audit table.
No crash on invalid input or database and file errors	NFR-08, FR-31	Exception hierarchy; one top-level handler in ConsoleApp; validation in the menu layer, the services and the database.
Simple for non-IT staff and secondary-school students	N1, CF4, NFR-10	Menu and Command objects, plain prompts, at most 3 menu levels, confirmation before final actions.
Testable and portable code	NFR-12, NFR-13	Interfaces for the clock, hasher, file store and repositories so that tests can replace them; CMake build; no platform-specific code outside the infrastructure classes.
Only necessary personal data is stored	NFR-14, Data Protection Act 2011	Minimal schema (Section 5); no date of birth, address or contact details.
About 51.7 hours for implementation	Phase 1 work breakdown	Thin services, table-driven life cycle, Should and Could features in separate classes that can be cut first.

2.2 Architectural style selected: layered architecture

The LMS is built as a layered architecture with three layers and a small set of infrastructure services. The layers and the rules between them are:

- **Presentation layer.** The console menus and prompts. It reads input, checks its type and range, shows results and asks for confirmation. It contains no business rules and no SQL.
- **Business logic layer.** The services that apply the business rules BR-01 to BR-15: authentication, assignments and extensions, submission, penalty calculation, marking, moderation, release and audit. It checks the user's permission in every method.
- **Data access layer.** The repository interfaces and their SQLite implementations. It is the only place where SQL text exists, and it uses prepared statements with bound parameters only.
- **Infrastructure services.** Small interfaces for things the business layer needs from the outside world: password hashing (libsodium), the managed submissions folder, and the system clock.

Three dependency rules keep the layers separate. A layer may call only the layer directly below it. The business layer depends on repository and infrastructure interfaces, never on SQLite or libsodium classes. The domain classes (User, Assignment, Submission, Mark and so on) are plain data-and-behaviour classes used by all layers, and they know nothing about the database.

2.3 Alternatives considered

Table 2.2: Architectural alternatives

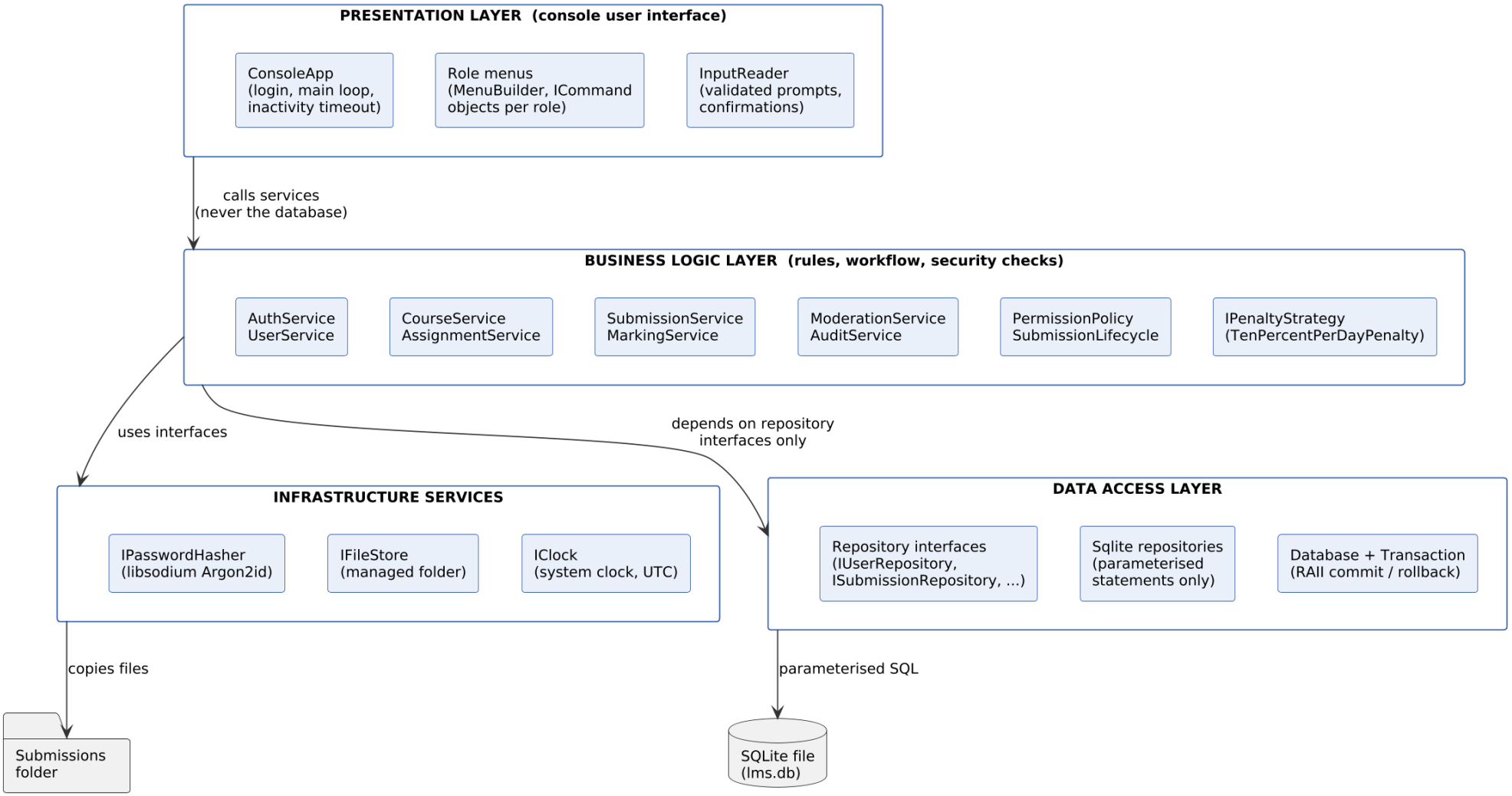
Option	Advantages	Problems	Decision
A. Layered (presentation, business logic, data access)	Matches the three layers named in SRS 4.2.1; each layer testable alone; SQL confined to one layer; simple to explain at the viva.	Some boilerplate: a call passes through three layers.	Selected

Option	Advantages	Problems	Decision
B. Model–View–Controller (MVC)	Well known; separates display from data.	MVC suits GUIs where several views observe one model and are updated by events. A console program has one view that prints text, so the controller and view merge into the menu code and business rules drift into it. The extra notification mechanism would cost hours without any benefit.	Rejected
C. Single layer: menu functions call SQL directly	Fastest to start; fewest classes.	SQL text in the menus (injection risk, NFR-06); rules duplicated in several menus; the business layer cannot be unit-tested, so the 70% coverage target (NFR-12) is not reachable.	Rejected
D. Client–server with a separate database server	Many users at once; central administration.	Needs a server and network, which breaks the offline requirement (NFR-04) and the old-computer constraint; far more administration for the school.	Rejected

Layered architecture was chosen because it gives the strongest protection for the least effort. The main trade-off is accepted knowingly: a request passes through three layers, but with 800 students and a local database this costs far less than the 1-second limit in NFR-02.

2.4 Component diagram

Figure 2.1 shows the layers, the components inside them and the allowed dependencies. Arrows point from the caller to the component that is called. There is no arrow from the presentation layer to the data access layer, which shows that menus cannot reach the database.



(NOTE: CLUADE AI assistance was used in construction the diagrams)Figure 2.1: Layered architecture and component diagram of the Moderated LMS

Table 2.3: Component responsibilities

Layer	Component	Responsibility	Requirements
Presentation	ConsoleApp	Login screen, main loop, inactivity timeout, top-level exception handler.	FR-01, FR-05, FR-31
Presentation	Role menus (MenuBuilder, Menu, ICommand)	Build the menu for the logged-in role; run the chosen command; at most 3 levels.	FR-01, FR-03, NFR-10
Presentation	InputReader	Read and validate type, range, length and allowed values; ask again on error; ask confirmation.	FR-31, FR-18
Business logic	AuthService, UserService	Login, lockout, password change, account management.	FR-01 to FR-07
Business logic	CourseService, AssignmentService	Departments, courses, lecturers, enrolment, assignments, extensions, effective deadline.	FR-08 to FR-14
Business logic	SubmissionService	Submit, resubmit, late submission, listing, released-mark view.	FR-14 to FR-20, FR-27
Business logic	MarkingService	Mark and re-mark with automatic penalty.	FR-19, FR-21, FR-22
Business logic	ModerationService	Review, approve, adjust, return, release, correct released marks.	FR-23 to FR-26, FR-35
Business logic	AuditService	Write and read audit records.	FR-29, FR-30
Business logic	PermissionPolicy, SubmissionLifecycle	Role permission matrix; allowed status changes.	FR-03, FR-28
Business logic	IPenaltyStrategy	Days late, penalty and final mark.	FR-19, BR-05 to BR-07
Data access	Repository interfaces and SQLite repositories	Read and write rows using parameterised statements.	NFR-06
Data access	Database, Statement, Transaction	Open the file, enable foreign keys, prepare statements, commit or roll back.	NFR-09
Infrastructure	IPasswordHasher, IFileStore, IClock	Argon2id hashing; copy and validate files; read the time in UTC.	NFR-05, FR-16, BR-04

2.5 Cross-cutting design decisions

Some concerns affect every layer. Table 2.4 states how each one is handled so that the code is consistent.

Table 2.4: Cross-cutting design decisions

Concern	Design decision	Requirements
Passwords	LibsodiumArgon2idHasher uses the library's password-hash string function with the interactive cost limits, tuned on the oldest available machine so that login stays within 2 seconds. Each hash string contains its own random 16-byte salt and parameters. Only one hash is checked per login. For an unknown username a dummy hash is checked, so both failures take similar time and look the same.	NFR-01, NFR-05, FR-02, CF7
Authorisation	Two checks. The menu shows only the functions in User::permittedFunctions(). Every service method then calls PermissionPolicy::require() and checks scope (own courses, own department, own submissions). A refused call throws AuthorizationException and is written to event_log.	FR-03, NFR-07, BR-10, BR-15
Transactions	An operation that changes a mark, a status or an audit record opens one Transaction object. It commits at the end and rolls back automatically if an exception leaves the scope (RAII). Foreign keys are switched on in Database::open().	NFR-09, FR-29, FR-31
Time	IClock::now() is the only source of time. Times are held as UTC time points and stored as ISO-8601 text (for example 2026-11-02T14:30:00Z). They are converted to local time only for display. FixedClock lets tests set the time to the second.	BR-03 to BR-06, R13
Errors	Services throw LmsException subclasses (Figure 3.7). ConsoleApp catches them for each command, shows the user message and returns to the menu. Any other std::exception is logged without sensitive data and shown as a general error. The program never ends because of an error.	NFR-08, FR-31
Input validation	Three lines of defence: InputReader (type, range, length), services (business rules), database CHECK constraints and triggers (last resort).	FR-31, NFR-06

Concern	Design decision	Requirements
File handling	IFileStore validates the file (exists, not empty, 10 MB or less, extension PDF, DOCX, TXT, ZIP, PNG or JPG) and copies it to a path built by the system from the assignment, student and version numbers. A path typed by the user is only read, never used to write. The original file name is stored as text only.	FR-16, FR-15
Logging	event_log records access-denied events, failed and locked logins, and file or database errors. It never stores passwords, hashes or file contents.	FR-02, FR-03, NFR-05
Inactivity timeout	ConsoleApp calls Session::isExpired(now) before each command. After 10 minutes without input the session is discarded without saving any half-finished action.	FR-05

2.6 Design rationale

The architecture puts the security and integrity rules where they cannot be bypassed. Because menus can reach only services, and services can reach the database only through repositories, there is exactly one path to every piece of data. That path checks the role, validates the rule, uses a prepared statement and, for marks, writes the audit record in the same transaction. This is the main reason a layered style was preferred over a flatter design, even though it needs more classes.

The design also protects the schedule. The Must requirements are served by a small core of classes (AuthService, SubmissionService, MarkingService, ModerationService, AuditService and the repositories). The Should and Could features (FR-05, FR-07, FR-12, FR-25, FR-32 to FR-35) are separate methods or stand-alone classes that nothing else depends on, so they can be cut under the schedule-control rule without touching the Must code.

3. Class Design

3.1 Notation and conventions

The class diagrams use UML notation with C++17 types. Visibility is shown as + public, - private and # protected. `{abstract}` marks an abstract operation and the keyword “interface” marks a class with only pure virtual operations. `TimePoint` means `std::chrono::system_clock::time_point` in UTC. Multiplicities are written at both ends of an association. For readability the design is shown in seven figures, and class and operation names are identical in all of them. In the service signatures, the parameter `s` is the logged-in Session.

3.2 Domain model

Figure 3.1 shows the domain classes and how they relate. The main design choices are:

- **Enrolment** is an association class between Course and Student. Unenrolling sets `unenrolledAt` and never deletes the row, so existing submissions and marks stay valid (FR-10, BR-13).
- **Mark** is separate from Submission and is created when the lecturer first marks (0..1 per submission). It holds the raw mark, the penalty snapshot, the final mark and the moderation and release details.
- **Extension** keeps every extension granted. The effective deadline is not stored; it is calculated as the later of the assignment deadline and the latest extension for that student (BR-03). This avoids two copies of one fact.
- **SubmissionVersion** keeps every file version. The latest version is the one marked and earlier versions remain for the record (FR-17).
- **AuditRecord** belongs to a Submission and has no operations that change it after creation.

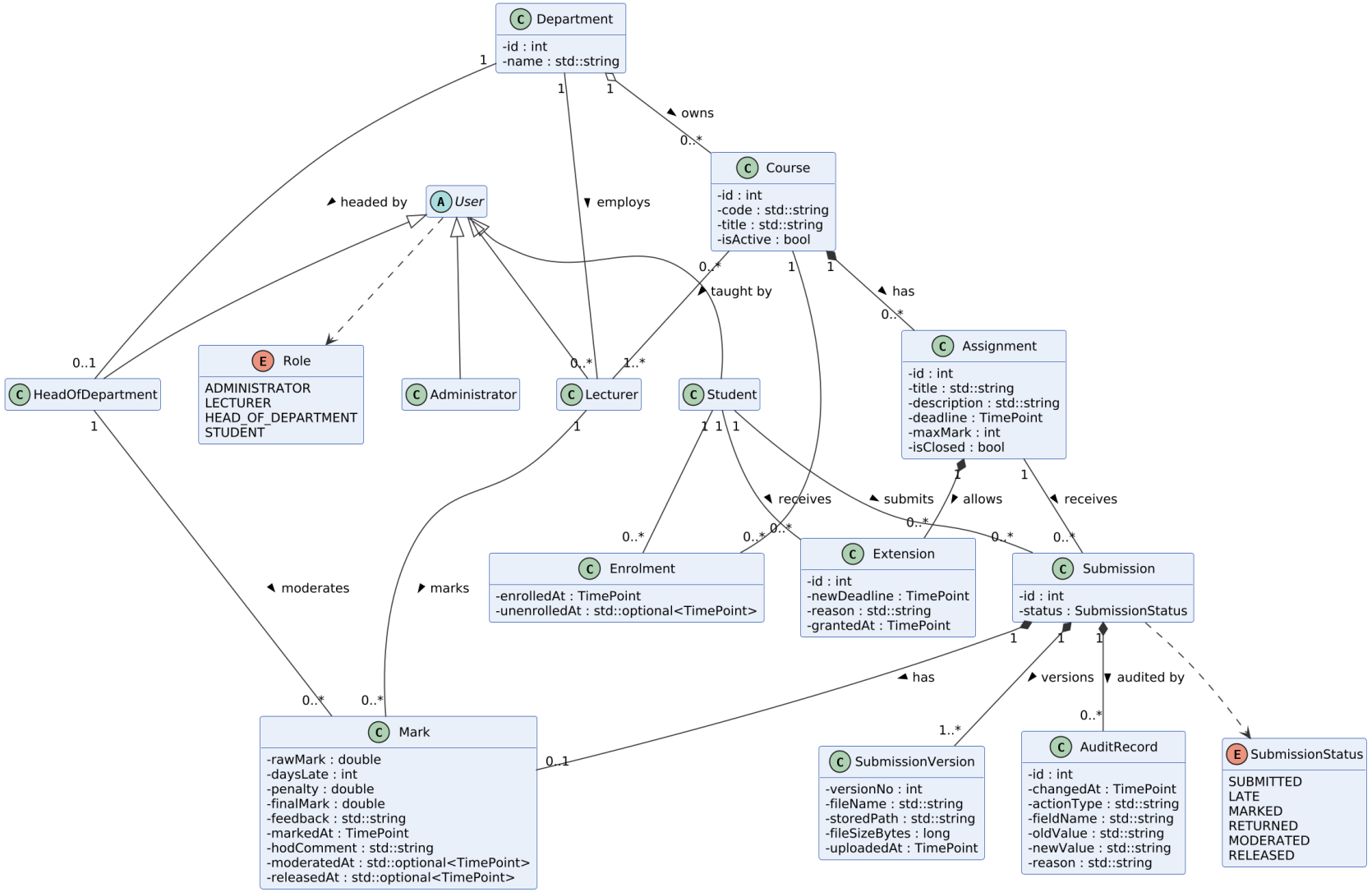


Figure 3.1: Domain class diagram

3.3 User hierarchy and Factory

User is an abstract class. It holds the data every account has and declares three abstract operations: `getRole()`, `permittedFunctions()` and `menuTitle()`. Administrator, Lecturer, HeadOfDepartment and Student override them. This is polymorphism in use: the menu builder and the permission policy work with a `User&` and never test the role with an if-chain. BR-01 (one role per account) is satisfied because each object is exactly one subclass.

The Function enumeration names every action in the permission matrix of the SRS. For example, `permittedFunctions()` of a Student returns VIEW_COURSES, SUBMIT_WORK and VIEW_RELEASED_MARKS, and it does not return MARK_SUBMISSION, so no marking menu item is ever created for a student. UserFactory is the only place that creates user objects: it reads the role column of a database record and returns the right subclass.

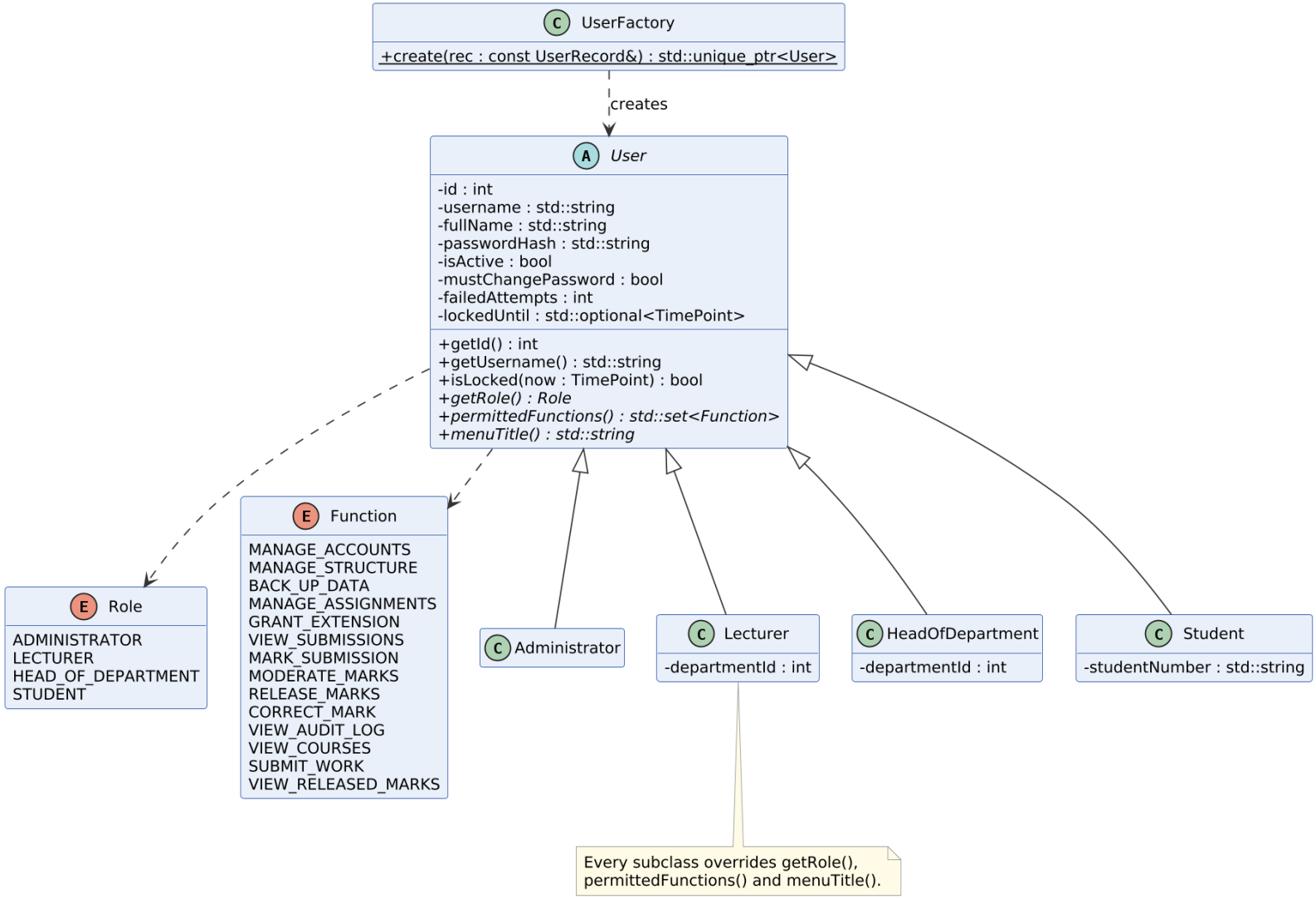


Figure 3.2: User hierarchy, Role and Function enumerations, and UserFactory

3.4 Session, permissions and menu commands

A Session holds the logged-in user and the time of the last input. The MenuBuilder reads `permittedFunctions()` and creates a Menu that contains only the commands whose `requiredFunction()` is in that set. Every menu entry is an ICommand object that validates its input and calls one service method. Menus nest to at most three levels (NFR-10). PermissionPolicy is the second, independent check used by the services (Section 2.5).

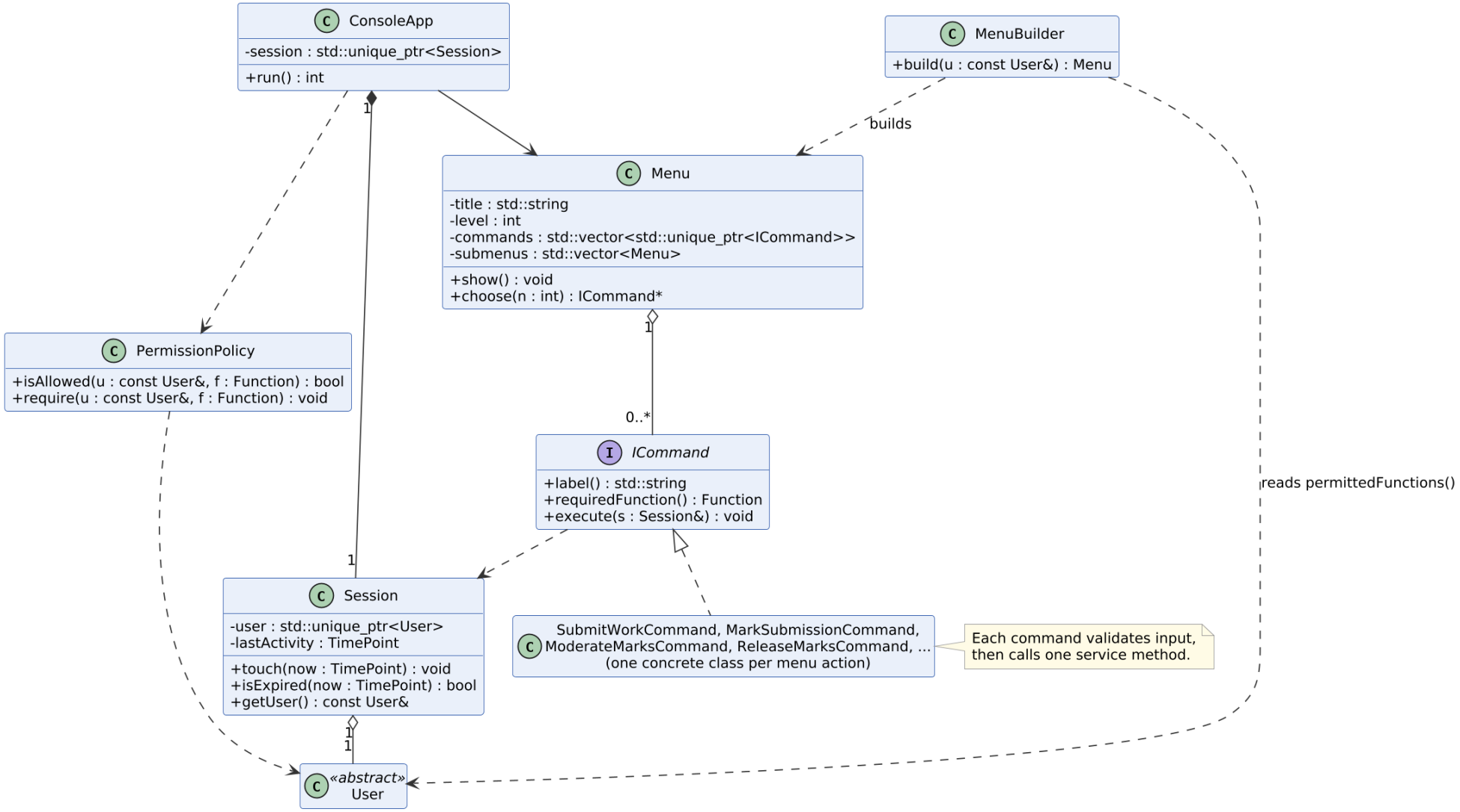


Figure 3.3: Session, permission policy and menu commands

3.5 Business logic services

Figure 3.4 shows the services and the objects they use. Each service has one area of responsibility (Table 3.1). The Strategy interface IPenaltyStrategy and the transition table SubmissionLifecycle are shared by the services that mark and submit work, so the rules exist once.

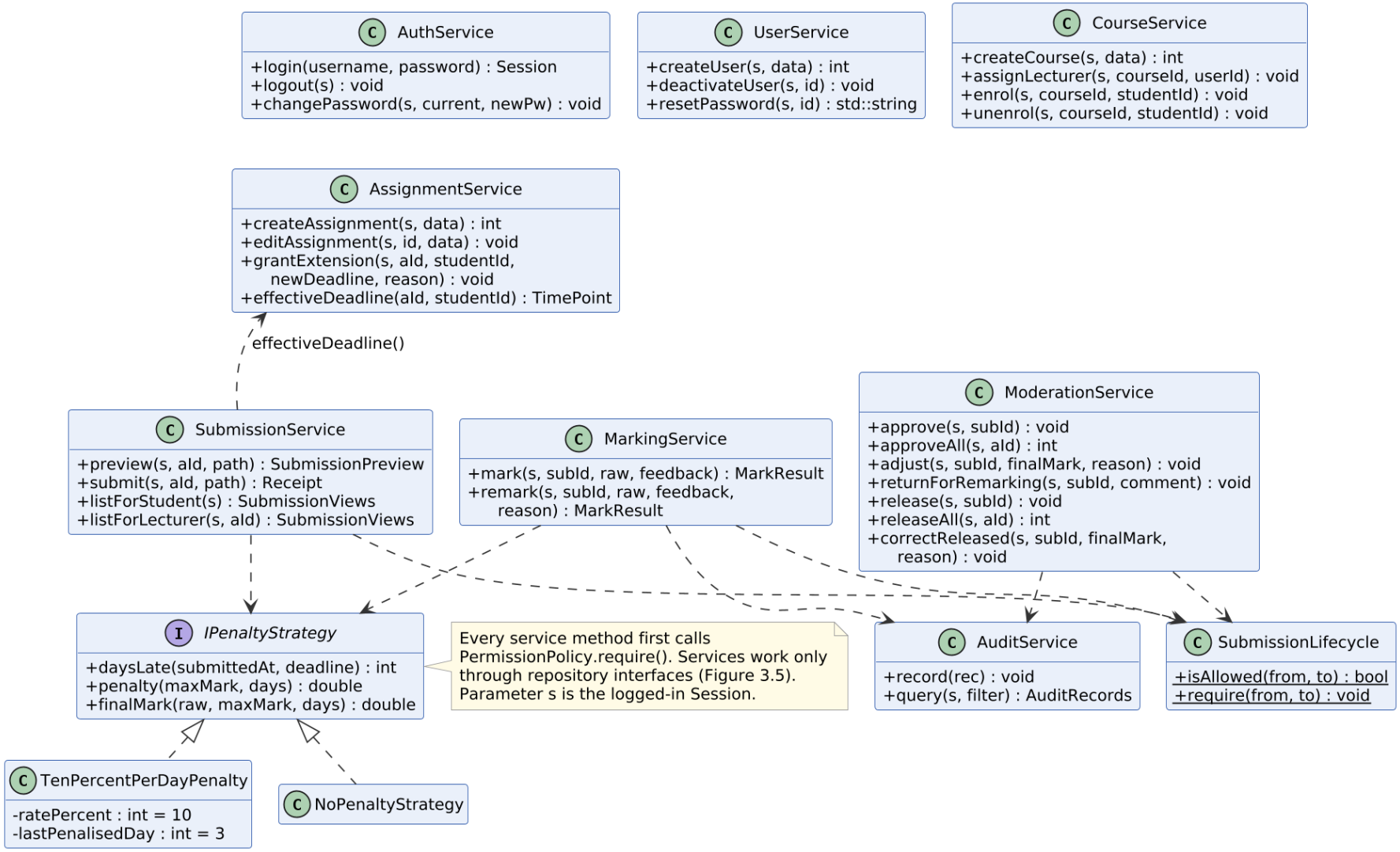


Figure 3.4: Business logic services, penalty strategy and submission life cycle

Table 3.1: Service responsibilities

Class	Responsibility	Requirements	Priority
AuthService	Login with lockout after 5 failures for 5 minutes; same message for wrong user or password; change password; force change of temporary password; logout.	FR-01, FR-02, FR-04	Must
UserService	Create, edit and deactivate accounts; reset a password to a temporary one (never delete).	FR-06, FR-07	Must / Should
CourseService	Create and edit departments and courses; assign and remove lecturers; enrol and unenrol students.	FR-08 to FR-10	Must
AssignmentService	Create and edit assignments; grant an extension with a reason; calculate the effective deadline.	FR-11 to FR-13	Must / Should
SubmissionService	Check file and deadline; preview on-time or late result with warning; store version; list submissions; show a student only released marks.	FR-14 to FR-18, FR-20, FR-27	Must
MarkingService	Enter and change raw mark and feedback; calculate days late, penalty and final mark; write audit records.	FR-19, FR-21, FR-22	Must
ModerationService	List department marks; approve, adjust or return; batch approve; release; correct a released mark.	FR-23 to FR-26, FR-35	Must / Should
AuditService	Create audit records inside the caller's transaction; list records by assignment, student or date range.	FR-29, FR-30	Must
BackupService	Copy the database file and the submissions folder to a folder chosen by the administrator. Stand-alone class, not drawn in Figure 3.4.	FR-34	Should
ReportExporter	Write the released marks of one assignment to a CSV file. Stand-alone class, not drawn in Figure 3.4.	FR-33	Could

3.6 Data access and infrastructure

Each aggregate has its own repository interface (Figure 3.5). The SQLite implementations hold a reference to Database and use Statement objects with bound parameters, so user input is never joined into SQL text (NFR-06). IAuditRepository has an `append()` and a `find()` but no update or delete operation, which makes the audit log append-only at the code level (FR-29).

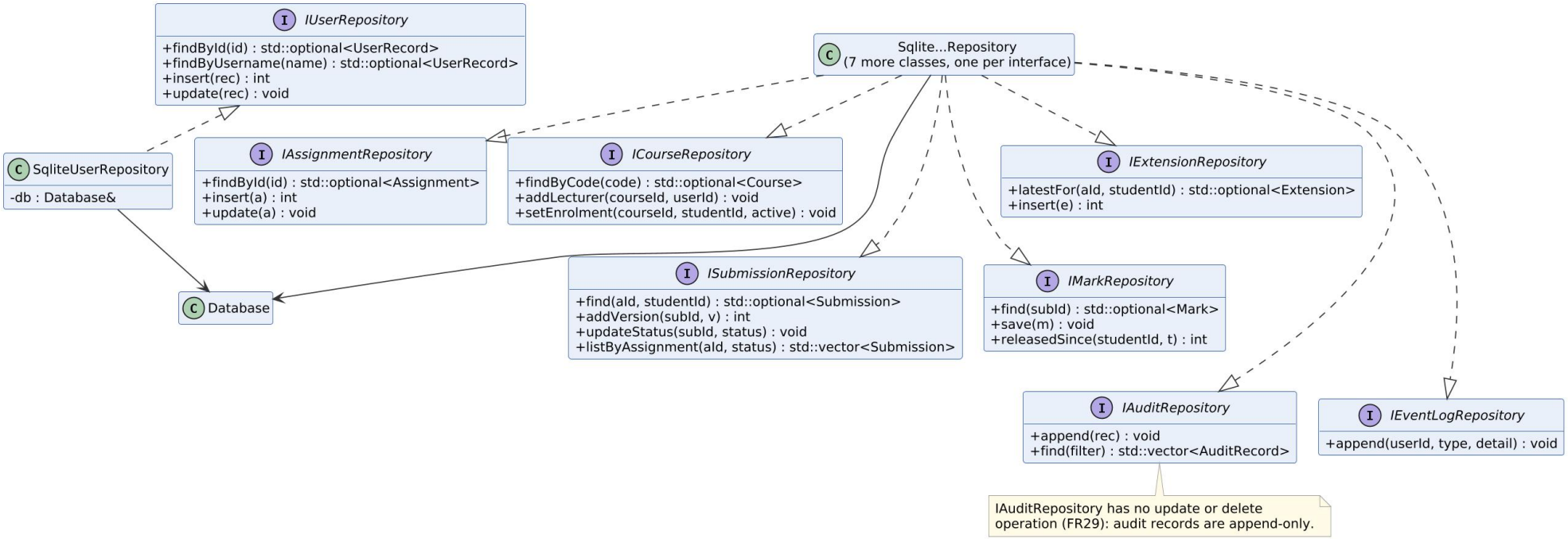


Figure 3.5: Repository interfaces and their SQLite implementations

Figure 3.6 shows the database helper classes and the three infrastructure interfaces. Transaction is a small RAII class: its destructor rolls back unless `commit()` was called, so an exception can never leave a half-saved mark. FixedClock and NoPenaltyStrategy exist so that tests can control time and isolate rules.

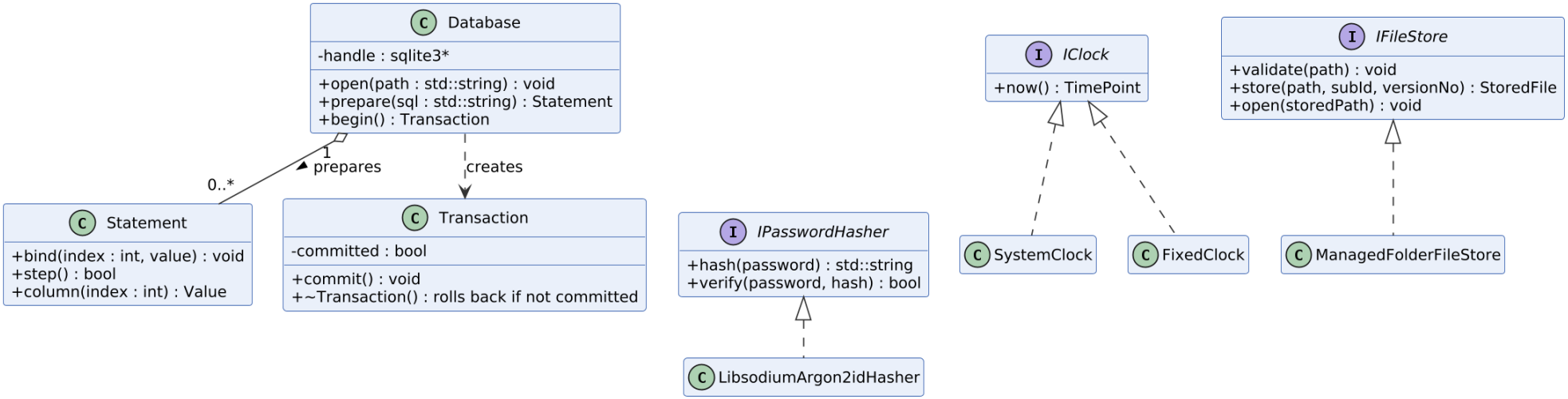


Figure 3.6: Database helper classes and infrastructure interfaces

3.7 Exception hierarchy

All errors raised by the application derive from LmsException (Figure 3.7). Table 3.2 states when each type is thrown and what the user sees.

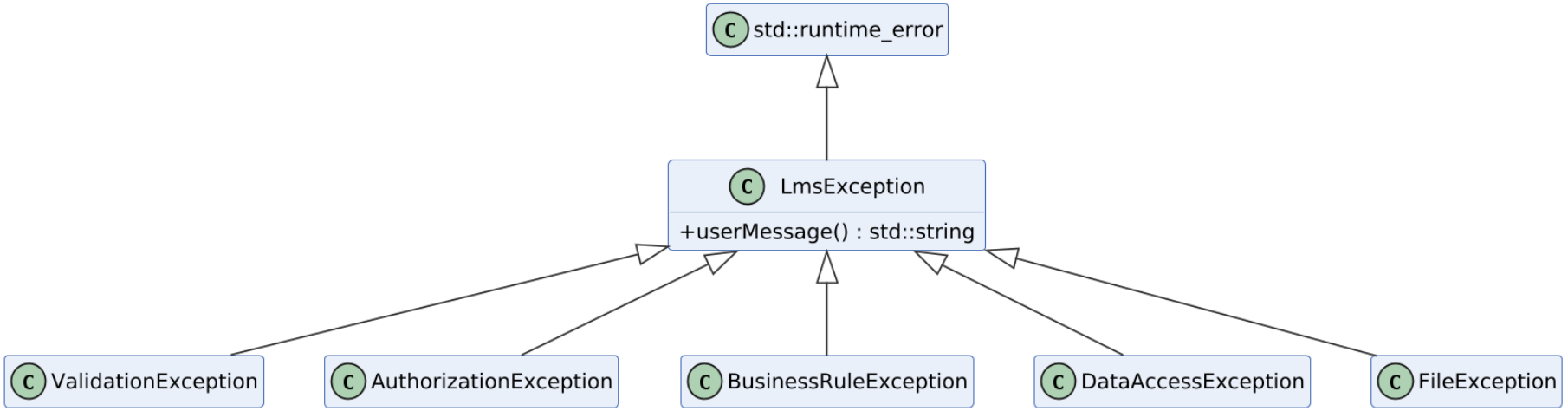


Figure 3.7: Exception hierarchy

Table 3.2: Exception types

Exception	Thrown when	User sees
ValidationException	Input is outside the allowed type, range, length or format.	Message naming the field and the allowed range; the prompt repeats.
AuthorizationException	A role tries a function or a record outside its permission or scope (FR-03, BR-10, BR-15).	“You are not allowed to do this.” The attempt is logged.
BusinessRuleException	A rule is broken, for example resubmitting after the deadline, an illegal status change (FR-28), or moderating one’s own mark.	The rule in plain words; nothing is changed.
DataAccessException	SQLite reports an error such as a locked or read-only database or a failed constraint.	General message; the transaction is rolled back.
FileException	A file is missing, empty, too large, of the wrong type, unreadable or the disk is full.	The reason; any existing submission is unchanged (FR-16).

3.8 Design patterns applied

Four design patterns are applied, together with two supporting idioms. Table 3.3 states why each was chosen and what was rejected.

Table 3.3: Design patterns and idioms

Pattern	Where it is used	Why it was chosen	Alternative rejected
Strategy	IPenaltyStrategy with TenPercentPerDayPenalty and NoPenaltyStrategy. Used by SubmissionService (warning) and MarkingService (calculation).	The late rule (10% per day for 3 days, then zero) is the rule most open to dispute and the main source of boundary errors (R13). Behind an interface it is one class with one set of boundary tests, and the warning shown to the student and the mark given by the lecturer always agree (FR-18, FR-19).	An if-else block inside MarkingService, which would be copied into SubmissionService and could drift apart.

Pattern	Where it is used	Why it was chosen	Alternative rejected
Repository (DAO)	Eight repository interfaces and Sqlite implementations.	Required by the brief: data only through a data-access layer. Keeps SQL and parameter binding in one layer and lets tests use in-memory fakes.	Active Record (entities save themselves), which mixes domain logic with SQL.
Factory (static factory method)	UserFactory::create() builds Administrator, Lecturer, HeadOfDepartment or Student from a database record.	The role is stored as text; one place converts it to the right subclass, so callers use only User and a new role needs a change in one place.	A switch on the role in every caller, which breaks the open-closed principle.
Command	ICommand objects held by Menu; one per menu action.	Each action carries its label and the Function it needs, so MenuBuilder can include or leave out items automatically. Adding an action does not change the menu loop.	Large switch statements inside each role's menu loop.
RAII Transaction (idiom)	Transaction in Database.	Commit or roll back is guaranteed even when an exception is thrown (NFR-09).	Manual begin, commit and rollback calls, which are easy to forget on an error path.
Dependency injection (idiom)	Services receive repository and infrastructure interfaces through their constructors; main() wires the real classes.	Needed for unit tests with FixedClock, fake repositories and NoPenaltyStrategy.	Global objects or singletons, which hide dependencies and block testing.
Table-driven life cycle (instead of GoF State)	SubmissionLifecycle holds the allowed transitions (Table 4.1).	The six statuses have the same data and differ only in which transitions are allowed. A State class for each would add six classes without state-specific behaviour. A table is shorter and can be checked by a loop test.	GoF State pattern with one class per status.

3.9 SOLID principles

Table 3.4: How the design follows the SOLID principles

Principle	How the design follows it	Example
Single responsibility	Each class has one reason to change.	MarkingService changes only when marking rules change; IPenaltyStrategy only when the late rule changes; AuditService only when audit content changes.
Open-closed	New behaviour is added by adding classes, not by editing tested ones.	A new late rule is a new IPenaltyStrategy; a new menu action is a new ICommand; a new role function adds one enumeration value.
Liskov substitution	Subclasses and implementations can replace their base without surprising callers.	Any User subclass works where User& is expected; SqliteUserRepository can be replaced by a fake; FixedClock can replace SystemClock.
Interface segregation	Interfaces are small and specific.	IClock has one operation; IAuditRepository has no update or delete; there is one repository per aggregate, not one large generic repository.
Dependency inversion	High-level rules depend on abstractions; concrete classes are chosen in one place.	Services use IUserRepository, IPasswordHasher and IClock; only main() names SqliteUserRepository and LibsodiumArgon2idHasher.

3.10 Key algorithm: effective deadline, days late and penalty

The deadline and penalty rules (BR-03 to BR-07) are specified here as exact steps so that the code and the boundary tests follow the same description.

- **Effective deadline** = the later of `assignment.deadline` and the latest `extensions.new_deadline` for that student and assignment (BR-03).
- **Late** if `submittedAt` is later than the effective deadline. A submission exactly at the deadline is on time (BR-04).
- **Days late** d = 0 if not late; otherwise the number of seconds late divided by 86,400 and rounded up to a whole number (BR-05).
- **Penalty** = 10% × maximum mark × d for d = 1, 2 or 3. For d of 4 or more the final mark is 0 (BR-06).

- **Final mark** = max(0, raw mark − penalty). Amounts are rounded half-up to 2 decimal places, because 10% of some maximum marks (for example 7) is not a multiple of 0.5.
- **HoD adjustment** replaces the final mark and is the only way to remove a penalty or the zero (BR-07).

Table 3.5 gives worked values for a maximum mark of 100 and a raw mark of 80. The rows at 24, 48 and 72 hours are the boundaries that Phase 5 will test.

Table 3.5: Worked penalty examples (maximum mark 100, raw mark 80)

Time of submission after the effective deadline	Days late d	Penalty	Final mark
Before the deadline, or exactly at it	0	0	80
1 second late	1	10	70
Exactly 24 hours late	1	10	70
24 hours and 1 second late	2	20	60
Exactly 72 hours late	3	30	50
72 hours and 1 second late	4	not applied	0

4. Behavioural Models

4.1 Submission life cycle (state machine)

A submission passes through six statuses: Submitted, Late, Marked, Returned, Moderated and Released. Figure 4.1 shows the allowed transitions and Table 4.1 gives the trigger, guard and effect of each one. This is the life-cycle table that FR-28 requires; SubmissionLifecycle holds exactly these transitions and refuses any other with a message.

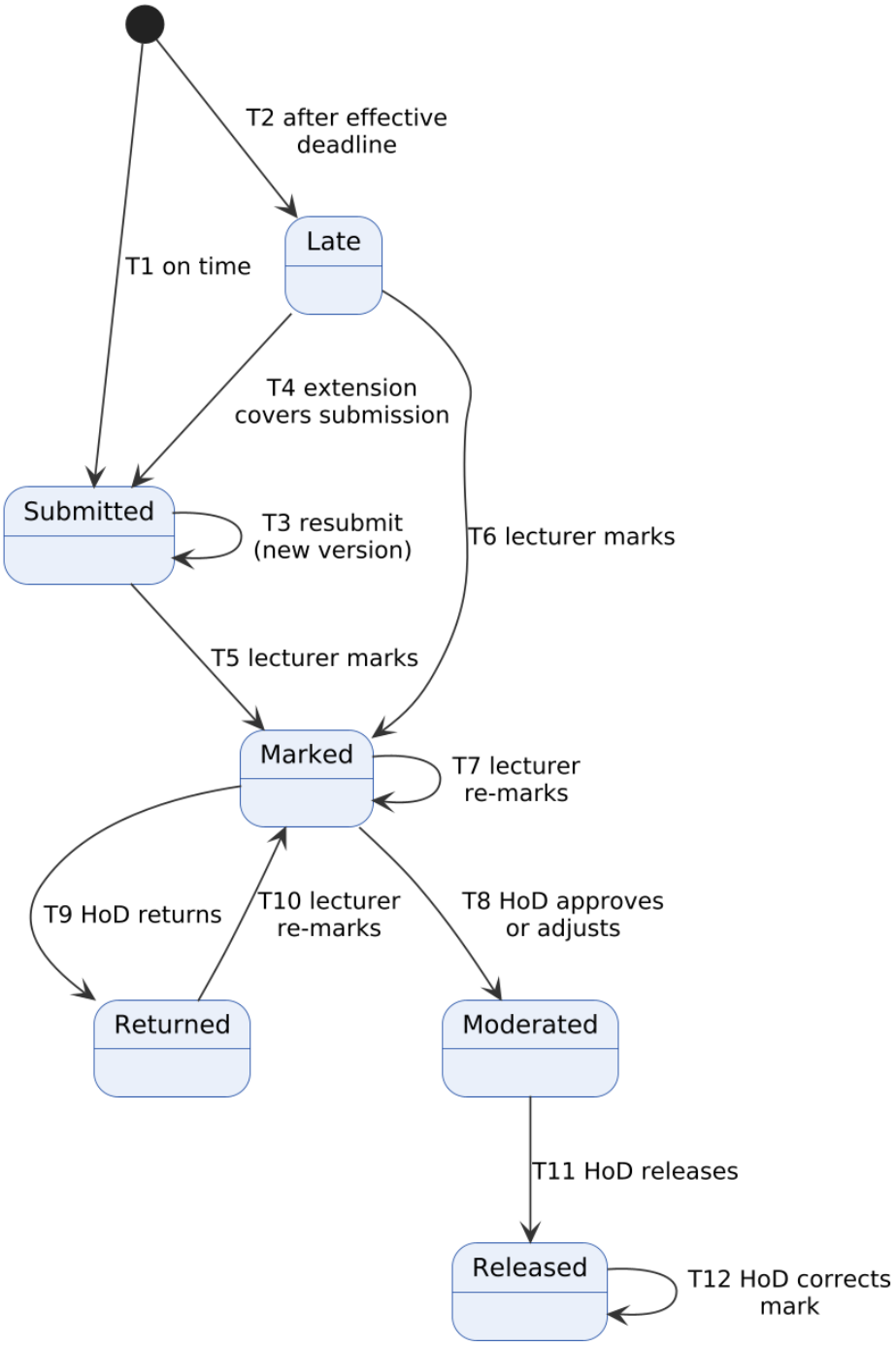


Figure 4.1: State machine diagram of the Submission life cycle

Table 4.1: Life-cycle transitions

ID	Transition	Trigger and actor	Guard	Effect	Source
T1	start → Submitted	Student submits the first file	Time is not after the effective deadline; file valid	Submission and version 1 created; receipt shown	FR-15, BR-04

ID	Transition	Trigger and actor	Guard	Effect	Source
T2	start → Late	Student submits the first file and confirms the warning	Time is after the effective deadline	Version 1 stored; days late and penalty shown first	FR-18, BR-05
T3	Submitted → Submitted	Student resubmits	Now is not after the effective deadline; not marked	New version added; earlier versions kept	FR-17, BR-08
T4	Late → Submitted	Lecturer grants an extension	Extension reason of at least 10 characters; submission time is not after the new deadline; not marked	Effective deadline moves later; status corrected	FR-13, BR-09
T5	Submitted → Marked	Lecturer marks	Raw mark 0 to maximum in steps of 0.5; feedback 10 to 2000 characters	Mark created; penalty 0; audit record “Initial marking”	FR-21, BR-12
T6	Late → Marked	Lecturer marks	Same as T5	Penalty applied (or zero after 3 days); audit record	FR-19, BR-06, BR-07
T7	Marked → Marked	Lecturer re-marks	Reason of at least 10 characters	Mark and feedback changed; audit record	FR-22
T8	Marked → Moderated	HoD approves or adjusts	HoD of the owning department; not the HoD’s own mark; adjust needs a reason	Moderation details saved; audit record if adjusted	FR-24, BR-10
T9	Marked → Returned	HoD returns for remarking	Comment entered	Comment saved	FR-24
T10	Returned → Marked	Lecturer re-marks	Reason of at least 10 characters	Mark changed; audit record	FR-22
T11	Moderated → Released	HoD releases	Status is Moderated; count confirmed for a batch	Release time and releaser saved	FR-26, BR-11

ID	Transition	Trigger and actor	Guard	Effect	Source
T12	Released → Released	HoD corrects the mark	Reason of at least 10 characters	Final mark changed; audit record; status stays Released	FR-35, BR-11

Any other change is refused with a BusinessRuleException. For example, a lecturer cannot change a Moderated or Released submission (FR-22), nothing can go from Marked to Released without moderation (BR-11), and a Late submission cannot be resubmitted (BR-08). Table 4.2 shows what a student can see in each status (FR-27, BR-02).

Table 4.2: Student view of each status

Status	Meaning	Student sees
Submitted	Received on time; may be replaced before the deadline.	Status “Submitted”; no mark or feedback.
Late	Received after the effective deadline.	Status “Late”; no mark or feedback.
Marked, Returned, Moderated	In marking or moderation.	“Awaiting release”; no mark or feedback.
Released	Approved and released by the HoD.	Mark, penalty and written feedback.

4.2 End-to-end process (activity diagram with swimlanes)

Figure 4.2 shows the whole process from the student’s submission to the released mark, with one swimlane for each of the Student, the System, the Lecturer and the Head of Department. Two loops are visible: the student is asked again until a valid file is given, and the mark returns to the lecturer when the HoD returns it for remarking. A late student can cancel at the warning, and then nothing is saved.

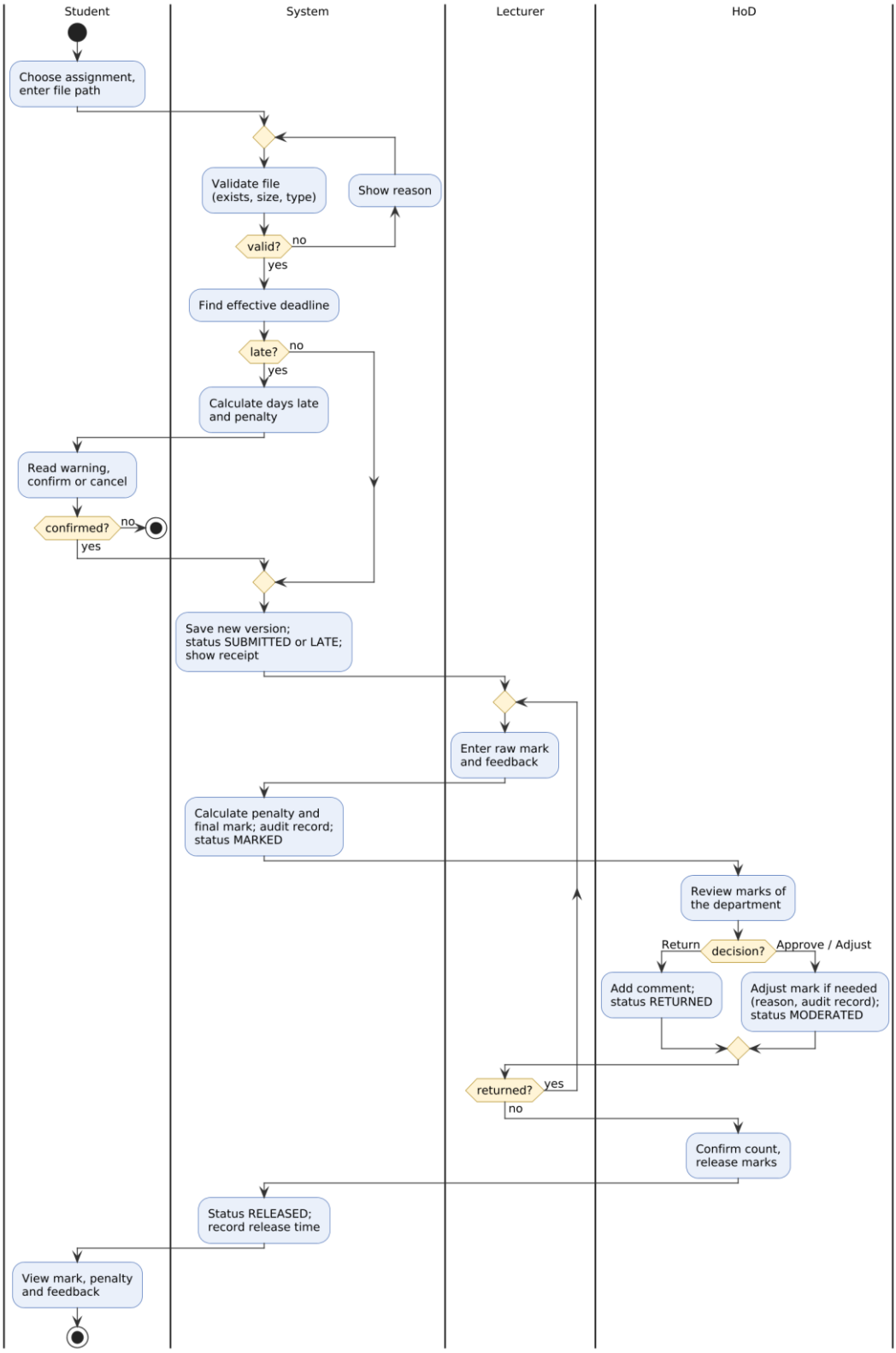


Figure 4.2: Activity diagram with swimlanes: from submission to released mark

4.3 Sequence diagram: Submit assignment

Figure 4.3 shows the “Submit assignment” use case, including the invalid-file, resubmission and late paths. The permission check, the clock and the transaction are shown as notes to

keep the lifelines readable. The service first calls `preview()` so the student sees the warning and gives confirmation before anything is saved (FR-18), and then calls `submit()`. Table 4.3 lists the paths so that Phase 5 can derive one test for each.

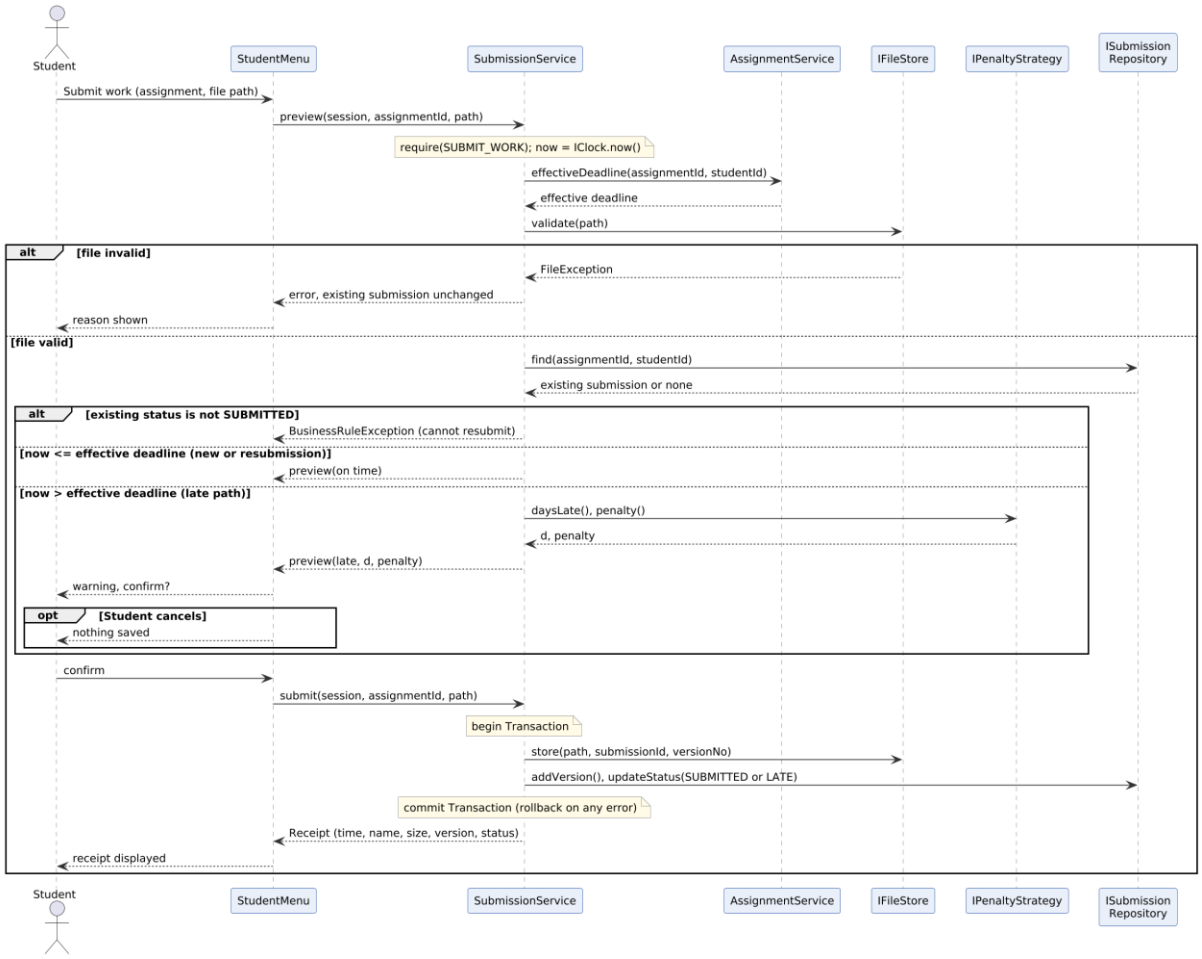


Figure 4.3: Sequence diagram: Submit assignment (on time, resubmission, late, invalid file)

Table 4.3: Paths through Submit assignment

Path	Condition	Outcome	Requirements
P1	First submission, valid file, now not after effective deadline	Status Submitted; version 1; receipt	FR-15, BR-04
P2	Existing submission with status Submitted, now not after effective deadline	New version added; earlier versions kept; receipt	FR-17, BR-08
P3	First submission after the effective deadline; student confirms	Status Late; warning with days late and penalty shown before saving	FR-18, BR-05, BR-06
P4	As P3, but the student cancels	Nothing saved	FR-18
P5	File missing, empty, over 10 MB or of a wrong type	Reason shown; existing submission unchanged	FR-16

Path	Condition	Outcome	Requirements
P6	Existing status is Late, Marked or later	Refused with a message; nothing changed	FR-17, FR-18, BR-08
P7	Disk full, locked database or other error while saving	Transaction rolled back; message; return to menu	FR-31, NFR-08

4.4 Sequence diagram: Moderate marks

Figure 4.4 shows how the HoD reviews marks and moderates one of them. The service checks that the HoD's department owns the course, that the HoD did not enter the mark (BR-10) and that the input is valid. It then checks the transition with SubmissionLifecycle and saves the change, the status and (for an adjustment) the audit record in one transaction.

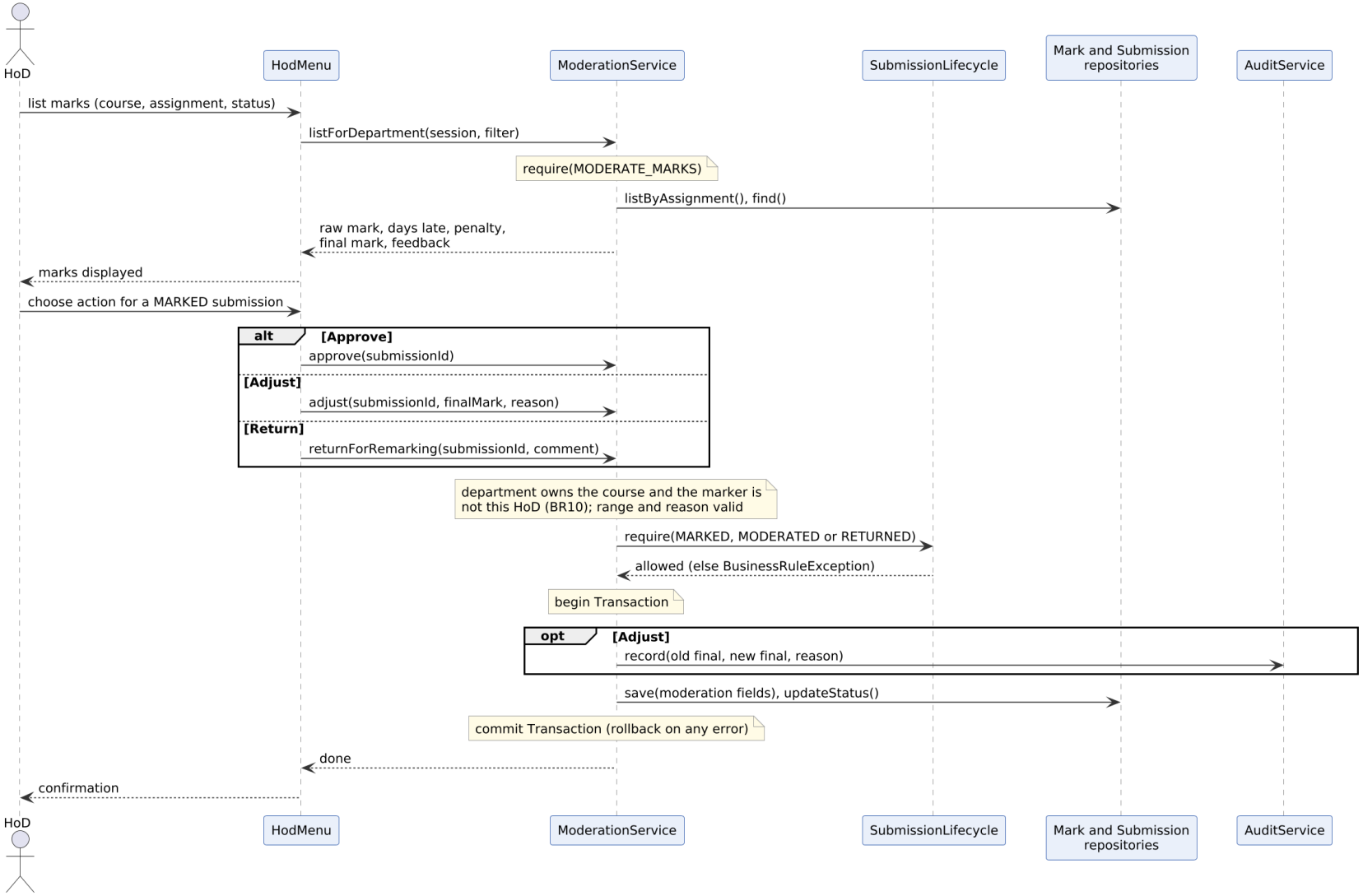


Figure 4.4: Sequence diagram: Moderate marks (approve, adjust or return)

4.5 Sequence diagram: Release marks

Figure 4.5 shows the release step. The HoD sees the number of moderated submissions and confirms before any mark is released, so no mark is released unseen (CF5). Release is the only action that moves a submission to Released (FR-26), and the student sees the mark only after that.

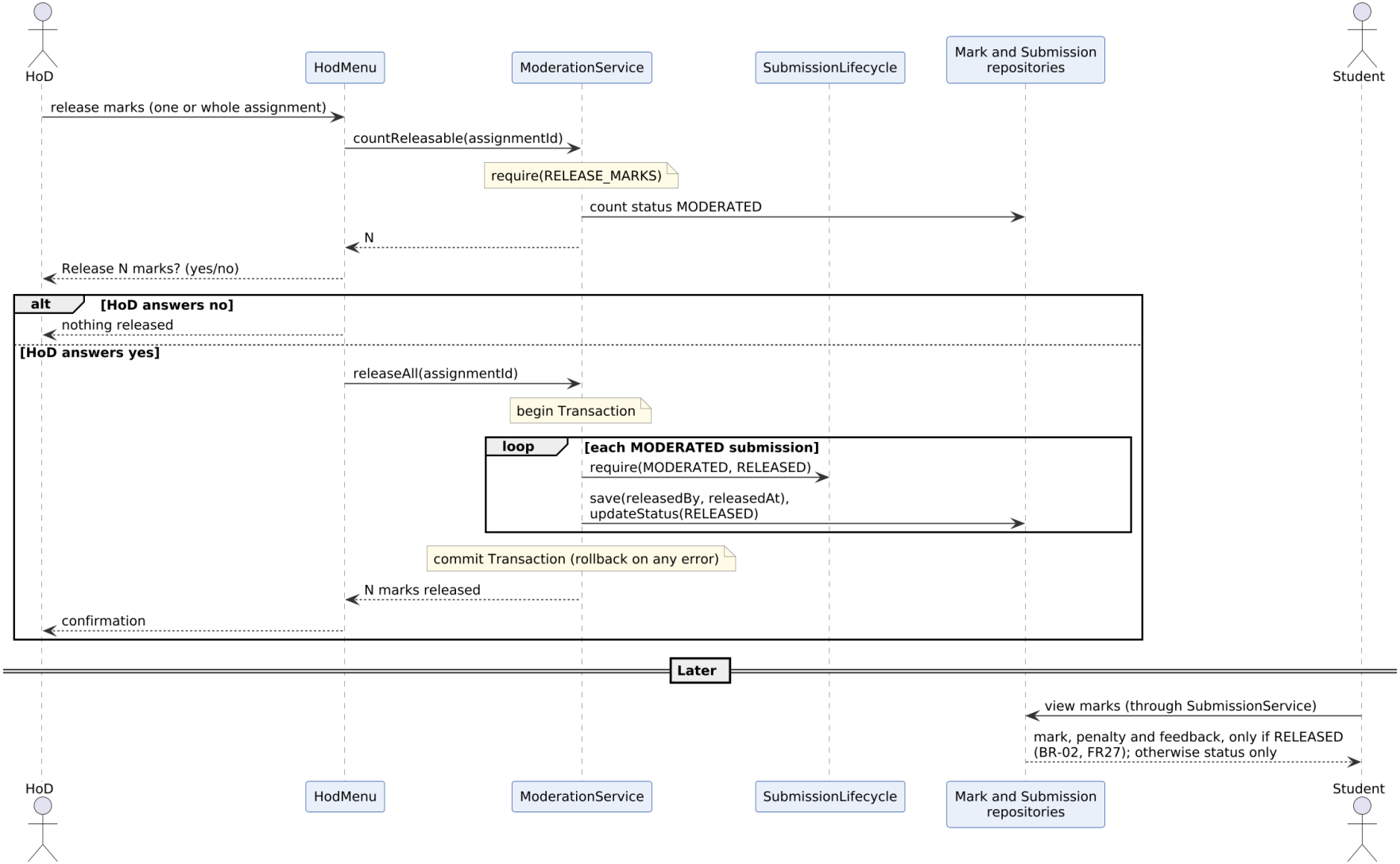


Figure 4.5: Sequence diagram: Release marks and the student view

5. Database Design

5.1 Design approach

- **SQLite 3**, one database file, opened only by the Database class. PRAGMA `foreign_keys = ON` is executed on every connection (NFR-09).
- **Surrogate keys.** Each table has an integer primary key, except link tables and marks, which use the key of their parent. Natural identifiers such as username, student number and course code are UNIQUE.
- **Times** are stored as UTC ISO-8601 text. **Booleans** are INTEGER 0 or 1. **Enumerations** are TEXT with a CHECK constraint, because the values are fixed by the business rules and a lookup table would only add joins.
- **No physical deletion** of accounts, courses, assignments, submissions or marks (BR-13). Rows are deactivated or closed, and delete triggers enforce this.
- **Minimal personal data** (NFR-14): full name, username, student number, role, department and password hash only.

5.2 Entity-relationship diagram

Figure 5.1 shows the twelve tables, and Table 5.1 states each relationship in words, because several lines connect to the users table.

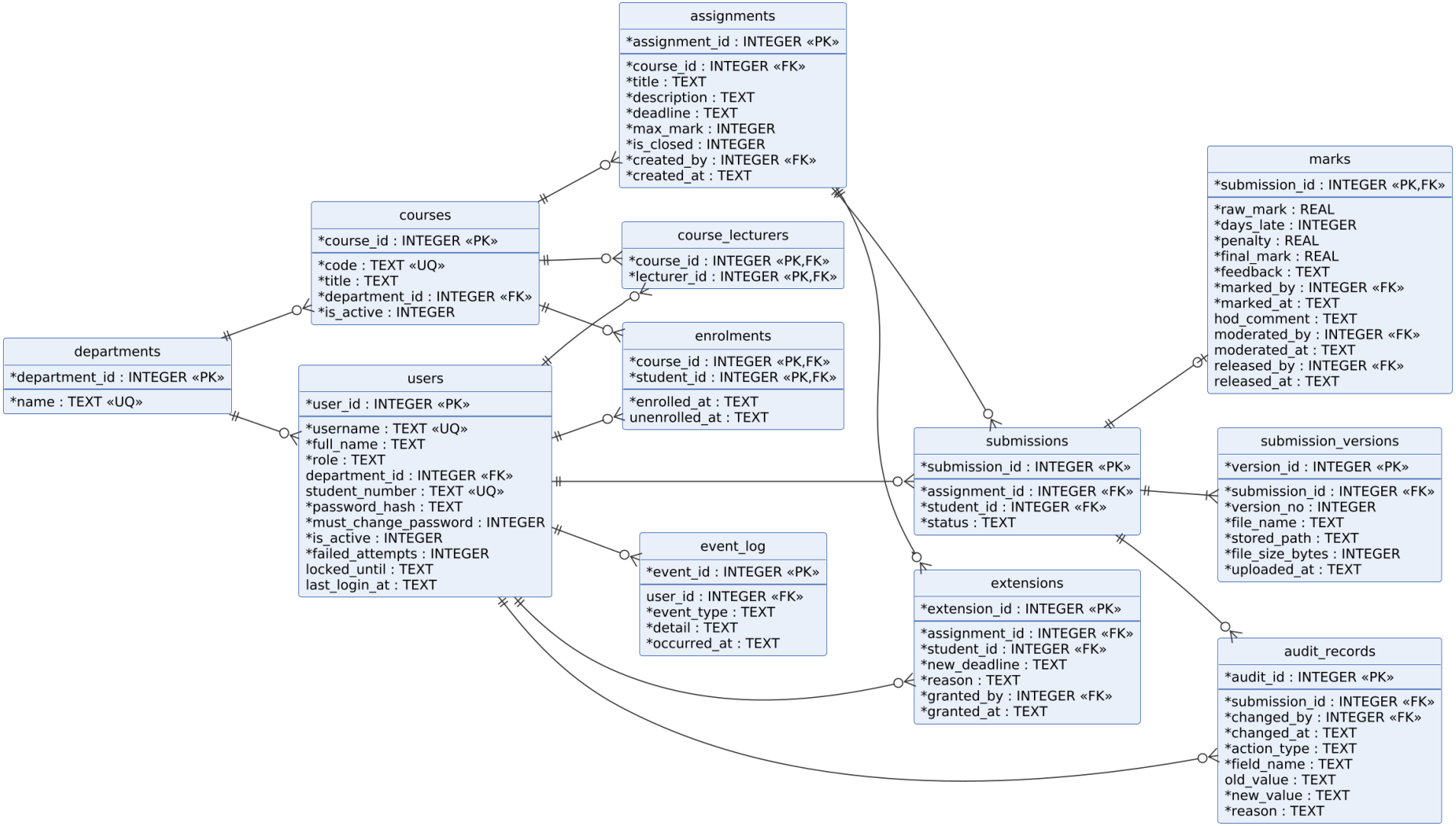


Figure 5.1: Entity-relationship diagram of the LMS database

Table 5.1: Relationships

Parent	Child	Cardinality	Foreign key and meaning
departments	users	1 : 0..*	users.department_id. Lecturers and HoDs belong to one department; at most one active HoD per department (unique partial index).
departments	courses	1 : 0..*	courses.department_id. Each course belongs to exactly one department.
courses, users	course_lecturers	1 : 0..* each	Link table for the many-to-many relation between courses and lecturers (FR-09).
courses, users	enrolments	1 : 0..* each	Link table for the many-to-many relation between courses and students, with enrolment dates (FR-10).
courses	assignments	1 : 0..*	assignments.course_id. users is also referenced by created_by.
assignments, users	extensions	1 : 0..* each	extensions.assignment_id and student_id; granted_by is the lecturer (FR-13).
assignments, users	submissions	1 : 0..* each	One submission per student per assignment (UNIQUE assignment_id, student_id).
submissions	submission_versions	1 : 1..*	Each submission has at least one file version (FR-15, FR-17).
submissions	marks	1 : 0..1	marks.submission_id is both primary key and foreign key. users is referenced three times (marked_by, moderated_by, released_by).
submissions, users	audit_records	1 : 0..* each	Who changed which submission's mark (FR-29).
users	event_log	1 : 0..*	Access-denied and login events; user_id may be empty for an unknown username.

5.3 Normalisation

All twelve tables are in Third Normal Form (3NF). Table 5.2 gives the argument for each table. In every table the columns are atomic (1NF), every non-key column depends on the whole key (2NF), and no non-key column depends on another non-key column (3NF).

Table 5.2: Normalisation argument

Table	Primary key	Argument
departments	department_id	Only name depends on the key.
users	user_id	All columns describe the account. department_id and student_number are empty when they do not apply to the role; this does not create a dependency between non-key columns because both depend only on user_id.
courses	course_id	The department's name is not repeated; only department_id is stored.
course_lecturers	(course_id, lecturer_id)	Key columns only; no non-key columns.
enrolments	(course_id, student_id)	Both dates describe the pair, not one part of it (full dependency).
assignments	assignment_id	Course details are not repeated; the maximum mark belongs to the assignment.
extensions	extension_id	Each row is one extension event; the effective deadline is not stored because it can be derived.
submissions	submission_id	Only the status is held here; file details are in submission_versions and mark details in marks.
submission_versions	version_id	File name, path, size and time depend on the version, not on the submission alone.
marks	submission_id	Every column describes the one mark of that submission.
audit_records	audit_id	Each row is one change event.
event_log	event_id	Each row is one event.

One deliberate redundancy. marks.days_late and marks.penalty can be calculated from the times and the maximum mark. They are stored as a snapshot at the moment of marking, so a later change to a deadline or to the rule cannot silently change a recorded mark, and so the HoD's list (FR-23) can be read without recalculating. Within the marks table neither column depends on another marks column (the maximum mark lives in assignments), so

3NF is not broken, but the redundancy is documented here and the penalty strategy remains the only code that produces these values. marks.final_mark is not derivable after a HoD adjustment, so it is a stored decision, not a copy.

5.4 Data dictionary

The tables below describe every column. “PK” is primary key, “FK” foreign key and “NN” not null. Time columns hold UTC ISO-8601 text.

Table 5.3: Data dictionary: departments

Column	Type	Key and constraints	Description
department_id	INTEGER	PK, autoincrement	Identifier of the department.
name	TEXT	NN, UNIQUE, 2 to 100 characters	Name of the department.

Table 5.4: Data dictionary: users

Column	Type	Key and constraints	Description
user_id	INTEGER	PK, autoincrement	Identifier of the account.
username	TEXT	NN, UNIQUE (case-insensitive), 3 to 30 characters	Login name (FR-06).
full_name	TEXT	NN, 2 to 100 characters	Full name of the person.
role	TEXT	NN; ADMINISTRATOR, LECTURER, HEAD_OF_DEPARTMENT or STUDENT	The one role of the account (BR-01); discriminator of the User hierarchy.
department_id	INTEGER	FK to departments; required for lecturer and HoD, empty otherwise	Department of a lecturer or HoD.
student_number	TEXT	UNIQUE; required for students, empty otherwise	Student number.
password_hash	TEXT	NN	Argon2id hash string including its random salt and parameters (NFR-05). Never the password.

Column	Type	Key and constraints	Description
must_change_password	INTEGER	NN, 0 or 1, default 1	True while a temporary password is in use (FR-04).
is_active	INTEGER	NN, 0 or 1, default 1	False for a deactivated account (FR-06).
failed_attempts	INTEGER	NN, 0 or more, default 0	Consecutive failed logins (FR-02).
locked_until	TEXT	Empty or a UTC time	End of the 5-minute lock (FR-02).
last_login_at	TEXT	Empty or a UTC time	Used to count marks released since the last login (FR-32).

Table 5.5: Data dictionary: courses

Column	Type	Key and constraints	Description
course_id	INTEGER	PK, autoincrement	Identifier of the course.
code	TEXT	NN, UNIQUE, 2 to 20 characters	Course code (FR-08).
title	TEXT	NN, 2 to 100 characters	Course title.
department_id	INTEGER	NN, FK to departments	The one department that owns the course.
is_active	INTEGER	NN, 0 or 1, default 1	False when the course is closed (BR-13).

Table 5.6: Data dictionary: course_lecturers

Column	Type	Key and constraints	Description
course_id	INTEGER	PK, FK to courses	The course.
lecturer_id	INTEGER	PK, FK to users	A user with the LECTURER role (checked by CourseService).

Table 5.7: Data dictionary: enrolments

Column	Type	Key and constraints	Description
course_id	INTEGER	PK, FK to courses	The course.

Column	Type	Key and constraints	Description
student_id	INTEGER	PK, FK to users	A user with the STUDENT role (checked by CourseService).
enrolled_at	TEXT	NN	Time of enrolment.
unenrolled_at	TEXT	Empty while enrolled	Time of unenrolment; the row is kept (FR-10).

Table 5.8: Data dictionary: assignments

Column	Type	Key and constraints	Description
assignment_id	INTEGER	PK, autoincrement	Identifier of the assignment.
course_id	INTEGER	NN, FK to courses	Course the assignment belongs to.
title	TEXT	NN, 2 to 100 characters	Title.
description	TEXT	NN, up to 2000 characters	Description.
deadline	TEXT	NN	Assignment deadline (UTC); can only move later (FR-12).
max_mark	INTEGER	NN, 1 to 100	Maximum mark (FR-11).
is_closed	INTEGER	NN, 0 or 1, default 0	True when closed to new submissions.
created_by	INTEGER	NN, FK to users	The lecturer who created it.
created_at	TEXT	NN	Creation time.

Table 5.9: Data dictionary: extensions

Column	Type	Key and constraints	Description
extension_id	INTEGER	PK, autoincrement	Identifier of the extension.
assignment_id	INTEGER	NN, FK to assignments	Assignment concerned.
student_id	INTEGER	NN, FK to users	Student who receives the extension.
new_deadline	TEXT	NN	New deadline (UTC); later than the current effective deadline (FR-13).
reason	TEXT	NN, at least 10 characters	Reason recorded for the HoD (CF2).

Column	Type	Key and constraints	Description
granted_by	INTEGER	NN, FK to users	The lecturer who granted it.
granted_at	TEXT	NN	Time granted.

Table 5.10: Data dictionary: submissions

Column	Type	Key and constraints	Description
submission_id	INTEGER	PK, autoincrement	Identifier of the submission.
assignment_id	INTEGER	NN, FK to assignments; UNIQUE with student_id	Assignment.
student_id	INTEGER	NN, FK to users	Student.
status	TEXT	NN; SUBMITTED, LATE, MARKED, RETURNED, MODERATED or RELEASED	Position in the life cycle (Table 4.1).

Table 5.11: Data dictionary: submission_versions

Column	Type	Key and constraints	Description
version_id	INTEGER	PK, autoincrement	Identifier of the version.
submission_id	INTEGER	NN, FK to submissions	Submission.
version_no	INTEGER	NN, 1 or more; UNIQUE with submission_id	Version number (FR-15).
file_name	TEXT	NN	Original file name, stored as text only.
stored_path	TEXT	NN	Path of the copy in the managed folder, built by the system.
file_size_bytes	INTEGER	NN, 1 to 10,485,760	Size of the file (FR-16).
uploaded_at	TEXT	NN	Time of submission; used for lateness (BR-04).

Table 5.12: Data dictionary: marks

Column	Type	Key and constraints	Description
submission_id	INTEGER	PK, FK to submissions	The marked submission.

Column	Type	Key and constraints	Description
raw_mark	REAL	NN, 0 or more, steps of 0.5, not above the maximum mark	Mark entered by the lecturer (BR-14).
days_late	INTEGER	NN, 0 or more, default 0	Snapshot of days late at marking (BR-05).
penalty	REAL	NN, 0 or more, default 0	Snapshot of the late penalty (BR-06).
final_mark	REAL	NN, 0 or more, not above the maximum mark	Mark the student sees after release (BR-06, BR-07).
feedback	TEXT	NN, 10 to 2000 characters	Written feedback (FR-21).
marked_by	INTEGER	NN, FK to users	Lecturer who marked.
marked_at	TEXT	NN	Time of the latest marking.
hod_comment	TEXT	Empty unless returned	HoD comment when returned for remarking (FR-24).
moderated_by	INTEGER	FK to users, empty until moderated	HoD who approved or adjusted.
moderated_at	TEXT	Empty until moderated	Time of moderation.
released_by	INTEGER	FK to users, empty until released	HoD who released.
released_at	TEXT	Empty until released	Time of release (FR-26).

Table 5.13: Data dictionary: audit_records

Column	Type	Key and constraints	Description
audit_id	INTEGER	PK, autoincrement	Identifier of the record.
submission_id	INTEGER	NN, FK to submissions	Submission whose mark changed.
changed_by	INTEGER	NN, FK to users	Who made the change.
changed_at	TEXT	NN	When (UTC).
action_type	TEXT	NN; INITIAL_MARKING, REMARK, MODERATION_ADJUSTMENT or POST_RELEASE_CORRECTION	Kind of change.

Column	Type	Key and constraints	Description
field_name	TEXT	NN; RAW_MARK, FEEDBACK or FINAL_MARK	Which value changed; one record per changed value.
old_value	TEXT	Empty for the first mark	Value before the change (BR-12).
new_value	TEXT	NN	Value after the change.
reason	TEXT	NN, at least 10 characters	Why (“Initial marking” for the first mark).

Table 5.14: Data dictionary: event_log

Column	Type	Key and constraints	Description
event_id	INTEGER	PK, autoincrement	Identifier of the event.
user_id	INTEGER	FK to users, may be empty	User concerned; empty for an unknown username.
event_type	TEXT	NN; ACCESS_DENIED, LOGIN_OK, LOGIN_FAILED, LOGIN_LOCKED, FILE_ERROR or DB_ERROR	Kind of event.
detail	TEXT	NN, up to 500 characters	Short description; never a password, hash or file content (NFR-05).
occurred_at	TEXT	NN	Time of the event.

5.5 SQL schema

The schema below is the single source of the database structure. It is kept in one versioned script, so that a change after Gate 2 is made in one place and goes through change control (risk R16).

```
PRAGMA foreign_keys = ON;   -- executed on every connection

CREATE TABLE departments (
  department_id  INTEGER PRIMARY KEY AUTOINCREMENT,
  name           TEXT NOT NULL UNIQUE CHECK (length(name) BETWEEN 2
AND 100)
);

CREATE TABLE users (
  user_id              INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
  username             TEXT NOT NULL UNIQUE COLLATE NOCASE
                       CHECK (length(username) BETWEEN 3 AND 30),
  full_name            TEXT NOT NULL CHECK (length(full_name)
BETWEEN 2 AND 100),
  role                 TEXT NOT NULL CHECK (role IN
('ADMINISTRATOR','LECTURER',

'HEAD_OF_DEPARTMENT','STUDENT')),
  department_id        INTEGER REFERENCES
departments(department_id),
  student_number       TEXT UNIQUE,
  password_hash        TEXT NOT NULL,
  must_change_password INTEGER NOT NULL DEFAULT 1 CHECK
(must_change_password IN (0,1)),
  is_active            INTEGER NOT NULL DEFAULT 1 CHECK (is_active
IN (0,1)),
  failed_attempts      INTEGER NOT NULL DEFAULT 0 CHECK
(failed_attempts >= 0),
  locked_until         TEXT,
  last_login_at        TEXT,
  CHECK ((role IN ('LECTURER','HEAD_OF_DEPARTMENT')
          AND department_id IS NOT NULL AND student_number IS NULL)
      OR (role = 'STUDENT' AND student_number IS NOT NULL AND
department_id IS NULL)
      OR (role = 'ADMINISTRATOR' AND department_id IS NULL AND
student_number IS NULL))
);
-- at most one active HoD per department (FR-08)
CREATE UNIQUE INDEX ux_one_active_hod ON users(department_id)
  WHERE role = 'HEAD_OF_DEPARTMENT' AND is_active = 1;

CREATE TABLE courses (
  course_id      INTEGER PRIMARY KEY AUTOINCREMENT,
  code           TEXT NOT NULL UNIQUE CHECK (length(code) BETWEEN 2
AND 20),
  title          TEXT NOT NULL CHECK (length(title) BETWEEN 2 AND
100),
  department_id  INTEGER NOT NULL REFERENCES
departments(department_id),
  is_active      INTEGER NOT NULL DEFAULT 1 CHECK (is_active IN
(0,1))
);

CREATE TABLE course_lecturers (
  course_id    INTEGER NOT NULL REFERENCES courses(course_id),
  lecturer_id  INTEGER NOT NULL REFERENCES users(user_id),
  PRIMARY KEY (course_id, lecturer_id)
);

CREATE TABLE enrolments (
  course_id      INTEGER NOT NULL REFERENCES courses(course_id),
  student_id     INTEGER NOT NULL REFERENCES users(user_id),
  enrolled_at    TEXT NOT NULL,
```

```
  unenrolled_at  TEXT,
  PRIMARY KEY (course_id, student_id)
);

CREATE TABLE assignments (
  assignment_id  INTEGER PRIMARY KEY AUTOINCREMENT,
  course_id      INTEGER NOT NULL REFERENCES courses(course_id),
  title          TEXT NOT NULL CHECK (length(title) BETWEEN 2 AND
100),
  description    TEXT NOT NULL DEFAULT '' CHECK
(length(description) <= 2000),
  deadline       TEXT NOT NULL,
  max_mark       INTEGER NOT NULL CHECK (max_mark BETWEEN 1 AND
100),
  is_closed      INTEGER NOT NULL DEFAULT 0 CHECK (is_closed IN
(0,1)),
  created_by     INTEGER NOT NULL REFERENCES users(user_id),
  created_at     TEXT NOT NULL
);

CREATE TABLE extensions (
  extension_id   INTEGER PRIMARY KEY AUTOINCREMENT,
  assignment_id  INTEGER NOT NULL REFERENCES
assignments(assignment_id),
  student_id     INTEGER NOT NULL REFERENCES users(user_id),
  new_deadline   TEXT NOT NULL,
  reason         TEXT NOT NULL CHECK (length(reason) >= 10),
  granted_by     INTEGER NOT NULL REFERENCES users(user_id),
  granted_at     TEXT NOT NULL
);

CREATE TABLE submissions (
  submission_id  INTEGER PRIMARY KEY AUTOINCREMENT,
  assignment_id  INTEGER NOT NULL REFERENCES
assignments(assignment_id),
  student_id     INTEGER NOT NULL REFERENCES users(user_id),
  status         TEXT NOT NULL CHECK (status IN
('SUBMITTED','LATE','MARKED',

'RETURNED','MODERATED','RELEASED')),
  UNIQUE (assignment_id, student_id)
);

CREATE TABLE submission_versions (
  version_id       INTEGER PRIMARY KEY AUTOINCREMENT,
  submission_id    INTEGER NOT NULL REFERENCES
submissions(submission_id),
  version_no       INTEGER NOT NULL CHECK (version_no >= 1),
  file_name        TEXT NOT NULL,
  stored_path      TEXT NOT NULL,
  file_size_bytes  INTEGER NOT NULL CHECK (file_size_bytes BETWEEN
1 AND 10485760),
  uploaded_at      TEXT NOT NULL,
```

```
  UNIQUE (submission_id, version_no)
);

CREATE TABLE marks (
  submission_id  INTEGER PRIMARY KEY REFERENCES
submissions(submission_id),
  raw_mark       REAL NOT NULL CHECK (raw_mark >= 0
                                      AND raw_mark * 2 =
CAST(raw_mark * 2 AS INTEGER)),
  days_late      INTEGER NOT NULL DEFAULT 0 CHECK (days_late >= 0),
  penalty        REAL NOT NULL DEFAULT 0 CHECK (penalty >= 0),
  final_mark     REAL NOT NULL CHECK (final_mark >= 0),
  feedback       TEXT NOT NULL CHECK (length(feedback) BETWEEN 10
AND 2000),
  marked_by      INTEGER NOT NULL REFERENCES users(user_id),
  marked_at      TEXT NOT NULL,
  hod_comment    TEXT,
  moderated_by   INTEGER REFERENCES users(user_id),
  moderated_at   TEXT,
  released_by    INTEGER REFERENCES users(user_id),
  released_at    TEXT
);

CREATE TABLE audit_records (
  audit_id       INTEGER PRIMARY KEY AUTOINCREMENT,
  submission_id  INTEGER NOT NULL REFERENCES
submissions(submission_id),
  changed_by     INTEGER NOT NULL REFERENCES users(user_id),
  changed_at     TEXT NOT NULL,
  action_type    TEXT NOT NULL CHECK (action_type IN
('INITIAL_MARKING','REMARK',

'MODERATION_ADJUSTMENT','POST_RELEASE_CORRECTION')),
  field_name     TEXT NOT NULL CHECK (field_name IN
('RAW_MARK','FEEDBACK','FINAL_MARK')),
  old_value      TEXT,
  new_value      TEXT NOT NULL,
  reason         TEXT NOT NULL CHECK (length(reason) >= 10)
);

CREATE TABLE event_log (
  event_id     INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id      INTEGER REFERENCES users(user_id),
  event_type   TEXT NOT NULL CHECK (event_type IN
('ACCESS_DENIED','LOGIN_OK',

'LOGIN_FAILED','LOGIN_LOCKED','FILE_ERROR','DB_ERROR')),
  detail       TEXT NOT NULL CHECK (length(detail) <= 500),
  occurred_at  TEXT NOT NULL
);

-- indexes (Section 5.7)
CREATE INDEX ix_courses_department   ON courses(department_id);
```

```
CREATE INDEX ix_clecturers_lecturer  ON
course_lecturers(lecturer_id);
CREATE INDEX ix_enrol_student        ON enrolments(student_id);
CREATE INDEX ix_assignments_course   ON assignments(course_id);
CREATE INDEX ix_extensions_lookup    ON extensions(assignment_id,
student_id);
CREATE INDEX ix_sub_assign_status    ON submissions(assignment_id,
status);
CREATE INDEX ix_sub_student          ON submissions(student_id);
CREATE INDEX ix_marks_released       ON marks(released_at);
CREATE INDEX ix_audit_submission     ON
audit_records(submission_id, changed_at);

-- audit records are append-only (FR-29)
CREATE TRIGGER trg_audit_no_update BEFORE UPDATE ON audit_records
BEGIN SELECT RAISE(ABORT, 'Audit records cannot be changed'); END;
CREATE TRIGGER trg_audit_no_delete BEFORE DELETE ON audit_records
BEGIN SELECT RAISE(ABORT, 'Audit records cannot be deleted'); END;

-- a mark may not exceed the maximum mark of its assignment
CREATE TRIGGER trg_marks_max_insert BEFORE INSERT ON marks
WHEN NEW.raw_mark > (SELECT a.max_mark FROM submissions s
       JOIN assignments a ON a.assignment_id = s.assignment_id
       WHERE s.submission_id = NEW.submission_id)
  OR NEW.final_mark > (SELECT a.max_mark FROM submissions s
       JOIN assignments a ON a.assignment_id = s.assignment_id
       WHERE s.submission_id = NEW.submission_id)
BEGIN SELECT RAISE(ABORT, 'Mark is above the maximum mark'); END;
CREATE TRIGGER trg_marks_max_update BEFORE UPDATE OF raw_mark,
final_mark ON marks
WHEN NEW.raw_mark > (SELECT a.max_mark FROM submissions s
       JOIN assignments a ON a.assignment_id = s.assignment_id
       WHERE s.submission_id = NEW.submission_id)
  OR NEW.final_mark > (SELECT a.max_mark FROM submissions s
       JOIN assignments a ON a.assignment_id = s.assignment_id
       WHERE s.submission_id = NEW.submission_id)
BEGIN SELECT RAISE(ABORT, 'Mark is above the maximum mark'); END;

-- accounts, courses, assignments, submissions and marks are never
deleted (BR-13)
CREATE TRIGGER trg_users_no_delete BEFORE DELETE ON users
BEGIN SELECT RAISE(ABORT, 'Deactivate instead of deleting'); END;
CREATE TRIGGER trg_courses_no_delete BEFORE DELETE ON courses
BEGIN SELECT RAISE(ABORT, 'Deactivate instead of deleting'); END;
CREATE TRIGGER trg_assignments_no_delete BEFORE DELETE ON
assignments
BEGIN SELECT RAISE(ABORT, 'Close instead of deleting'); END;
CREATE TRIGGER trg_submissions_no_delete BEFORE DELETE ON
submissions
BEGIN SELECT RAISE(ABORT, 'Submissions are never deleted'); END;
CREATE TRIGGER trg_marks_no_delete BEFORE DELETE ON marks
BEGIN SELECT RAISE(ABORT, 'Marks are never deleted'); END;
```

5.6 Mapping from classes to tables

Table 5.15 shows how the class diagrams map to the tables. The design models behaviour with classes and enumerations, so some classes are not stored, and a few tables look different from their classes for the reasons in the last column.

Table 5.15: Mapping of classes to tables and justified differences

Class or enumeration	Table	Difference and justification
User, Administrator, Lecturer, HeadOfDepartment, Student	users	Single-table inheritance: the role column is the discriminator and UserFactory rebuilds the subclass. Four tables would repeat almost the same columns and need joins for login (DD-10). CHECK constraints keep department_id and student_number consistent with the role.
Department	departments	Direct mapping. The HoD is not stored in departments, which avoids a circular reference with users; a unique partial index enforces one active HoD.
Course	courses	Direct mapping.
Course–Lecturer association	course_lecturers	Many-to-many relation becomes a link table.
Enrolment (association class)	enrolments	Association class becomes a link table with attributes.
Assignment	assignments	Direct mapping. The effective deadline is a derived value and has no column.
Extension	extensions	Direct mapping. Every extension is kept as history.
Submission	submissions	Holds only identity, links and status. Files and marks are separate tables.
SubmissionVersion	submission_versions	Direct mapping. The latest version is the highest version_no.
Mark	marks	Separate table with the submission key as its key (1:0..1), so unmarked submissions have no empty mark columns (DD-09).

Class or enumeration	Table	Difference and justification
AuditRecord	audit_records	One row per changed value (raw mark, feedback or final mark) so that old and new values stay simple text. Append-only by triggers.
(event record written by AuthService and PermissionPolicy)	event_log	A technical record, not a domain class; it is written through IEventLogRepository.
Role, SubmissionStatus, Function (enumerations)	CHECK constraints	Role and status are fixed by the business rules, so CHECK constraints are used instead of lookup tables. Function is a program value and is never stored.
Session, Menu, ICommand, services, strategies, repositories, infrastructure classes	none	Run-time objects only; nothing to store. Session state is kept in memory and discarded at logout or timeout.

5.7 Indexes and performance

The performance targets of NFR-01 and NFR-02 assume 800 students, 50 staff and 20,000 submissions. At that size the database holds roughly 20,000 submissions, 30,000 versions, 20,000 marks and about 60,000 audit rows, which is a file of a few tens of megabytes. Table 5.16 lists the indexes that keep the common queries on indexed lookups instead of full scans.

Table 5.16: Indexes

Index	Supports	Requirement
users.username (UNIQUE)	Login lookup by username.	NFR-01
ix_enrol_student	A student’s course list.	FR-14
ix_clecturers_lecturer	A lecturer’s course list.	FR-20
ix_courses_department	A HoD’s list of department courses.	FR-23
ix_assignments_course	Assignments of a course.	FR-14
ix_extensions_lookup	Effective deadline of one student for one assignment.	FR-13, BR-03
ix_sub_assign_status	Submissions of an assignment, filtered by status; the HoD’s list of up to 500 submissions.	FR-20, FR-23, NFR-02

Index	Supports	Requirement
ix_sub_student	A student's own submissions.	FR-14
ix_marks_released	Count of marks released since the last login.	FR-32
ix_audit_submission	Audit records by submission and date.	FR-30

5.8 Where each rule is enforced

Important rules are enforced in more than one place, so that a mistake in one layer does not corrupt the data. Table 5.17 lists them.

Table 5.17: Defence in depth for key rules

Rule	Service or code	Database
Mark between 0 and the maximum, in steps of 0.5 (BR-14)	InputReader and MarkingService	CHECK on raw_mark; triggers against max_mark
Password never stored in clear (NFR-05)	IPasswordHasher is the only way to create password_hash	Column holds only the hash string
Student sees marks only when Released (BR-02)	SubmissionService returns mark fields only for status RELEASED	Not enforced by the database; covered by tests
Only allowed status changes (FR-28)	SubmissionLifecycle::require()	CHECK on the list of statuses
Audit record with every mark change (FR-29)	Same Transaction in MarkingService and ModerationService	Foreign key; append-only triggers
One active HoD per department (FR-08)	UserService	Unique partial index ux_one_active_hod
One submission per student per assignment	SubmissionService	UNIQUE (assignment_id, student_id)
No deletion of history (BR-13)	No delete operation exists in the services	BEFORE DELETE triggers
No SQL injection (NFR-06)	Statement binds parameters; no string joining	Not applicable

6. Traceability from Requirements to Design

Table 6.1 links each functional requirement to its use case, the main classes, the tables and the figures that show it. This is the design part of the traceability matrix; source files and test cases will be added in Phases 4 and 5 (Appendix A).

Table 6.1: Functional requirements traced to the design

FR	Use case (Phase 2)	Main classes	Tables	Figures
FR-01	Log in	AuthService, Session, MenuBuilder, User subclasses	users	3.2, 3.3
FR-02	Log in; Lock account	AuthService, IPasswordHasher	users, event_log	3.4
FR-03	All use cases	PermissionPolicy, MenuBuilder, User::permittedFunctions	event_log	3.2, 3.3
FR-04	Change password	AuthService, IPasswordHasher	users	3.4
FR-05	Log in	Session, ConsoleApp	none	3.3
FR-06	Manage user accounts	UserService, UserFactory	users	3.2, 3.4
FR-07	Manage user accounts	UserService	users	3.4
FR-08	Manage departments and courses	CourseService	departments, courses, users	3.4
FR-09	Assign lecturer to course	CourseService	course_lecturers	3.4
FR-10	Enrol student	CourseService	enrolments	3.4
FR-11	Manage assignments	AssignmentService	assignments	3.4
FR-12	Manage assignments	AssignmentService	assignments, submissions	3.4
FR-13	Grant individual extension	AssignmentService	extensions	3.4, 4.1

FR	Use case (Phase 2)	Main classes	Tables	Figures
FR-14	View courses and assignments	SubmissionService, AssignmentService	courses, enrolments, assignments, extensions, submissions	3.4
FR-15	Submit assignment	SubmissionService, IFileStore	submissions, submission_versions	4.3
FR-16	Validate submission file	IFileStore, FileException	none (existing rows unchanged)	4.3
FR-17	Resubmit before deadline	SubmissionService, SubmissionLifecycle	submission_versions, submissions	4.1, 4.3
FR-18	Submit late work	SubmissionService, IPenaltyStrategy	submissions	4.1, 4.3
FR-19	Mark submission	MarkingService, IPenaltyStrategy	marks	3.4, 4.2
FR-20	View submissions	SubmissionService, IFileStore	submissions, submission_versions	3.4
FR-21	Mark submission; Write feedback	MarkingService	marks, submissions	4.2
FR-22	Mark submission (re-mark)	MarkingService, AuditService	marks, audit_records	4.1
FR-23	Review department marks	ModerationService	submissions, marks, assignments, courses	4.4
FR-24	Moderate marks; Adjust; Return	ModerationService	marks, submissions, audit_records	4.4
FR-25	Moderate marks (batch)	ModerationService::approveAll	marks, submissions	4.4
FR-26	Release approved marks	ModerationService	marks, submissions	4.5
FR-27	View released marks and feedback	SubmissionService	marks, submissions	4.5
FR-28	All marking use cases	SubmissionLifecycle	submissions (status CHECK)	4.1
FR-29	Record audit entry	AuditService, IAuditRepository, Transaction	audit_records	3.5, 4.4
FR-30	View audit log	AuditService	audit_records	3.4

FR	Use case (Phase 2)	Main classes	Tables	Figures
FR-31	All use cases	InputReader, LmsException, Transaction	CHECK constraints	3.7, 3.6
FR-32	View released marks	SubmissionService, IMarkRepository	marks, users.last_login_at	3.5
FR-33	Export marks (Could)	ReportExporter	marks	Table 3.1
FR-34	Back up data	BackupService	database file and submissions folder	Table 3.1
FR-35	Correct released mark	ModerationService, AuditService	marks, audit_records	4.1

All Must requirements are covered by a class and, where data is stored, by a table. Business rules BR-01 to BR-15 appear in Table 4.1 (life cycle), Section 3.10 (deadline and penalty rules) and Table 5.17 (enforcement). The non-functional requirements are addressed by the drivers in Table 2.1 and the decisions in Table 2.4.

7. Gate 2 Review Checklist

Table 7.1 compares this document with the Phase 3 tasks of the assignment. It is the evidence for the supervisor at Gate 2.

Table 7.1: Gate 2 checklist

Check	Evidence	Status
Architectural style selected and justified; at least one rejected alternative	Sections 2.2 and 2.3 (three alternatives rejected)	Met
Component diagram and design rationale	Figure 2.1; Section 2.6	Met
Class diagram with visibility, types, multiplicities, inheritance and interfaces	Figures 3.1 to 3.7	Met
At least two design patterns, each justified	Section 3.8 (Strategy, Repository, Factory, Command)	Met
SOLID principles explained	Section 3.9	Met
State machine diagram of the Submission life cycle	Figure 4.1; Table 4.1	Met
Activity diagram with swimlanes, submission to released mark	Figure 4.2	Met
Sequence diagrams: Submit assignment (late and resubmission paths); Moderate and release marks	Figures 4.3, 4.4 and 4.5	Met
ERD normalised to at least 3NF	Figure 5.1; Table 5.2	Met
Data dictionary and SQL schema (tables, keys, constraints)	Section 5.4; Section 5.5	Met
Mapping of tables to classes with justified differences	Table 5.15	Met
Every functional requirement traced to the design	Table 6.1	Met
Names of classes, statuses and tables identical in all diagrams	Checked during the review of Section 4 against Sections 3 and 5	Met
Risk register updated at the gate	Section 8	Met

Recommendation. The design is complete and consistent with the Gate 1 requirements baseline, and it is ready for approval at Gate 2. After approval, any change to the architecture, class design, behavioural models or schema follows the change-control procedure of the project plan: the change, the reason, its effect on time and risk, and the decision are recorded.

8. Risk Register Update at Gate 2

The risk register is updated at each gate. Likelihood (L) and impact (I) are scored from 1 to 5, and exposure is L × I. A score of 15 or more is High, 8 to 14 is Medium and 7 or less is Low. Values in the table are the current values.

Table 8.1: Risk register at Gate 2

ID	Risk	L	I	Exp.	Level	Change at Gate 2	Mitigation	Owner
R1	Scope keeps changing	2	3	6	Low	Lowered: the design is built on the Gate 1 baseline and adds no new requirements.	Change control; remove Should items first.	Student
R2	Work takes longer than expected	4	4	16	High	Unchanged. Should and Could features are in separate methods or classes (Table 3.1) so they can be cut without touching Must code.	Weekly Friday check; cut list.	Student
R3	Illness, power cuts, computer failure	2	5	10	Medium	Unchanged. Diagram sources and this document are kept in Git.	Back up work; push to Git regularly.	Student
R4	Developer lacks some technical skills	3	3	9	Medium	Unchanged. libsodium and SQLite are hidden behind IPasswordHasher and Database, so a problem affects one class.	Prototype libsodium (Argon2id) and SQLite early in Phase 4.	Student
R5	Security problems	2	5	10	Medium	Unchanged. Design gives several checks for each key rule (Table 5.17).	Parameterised queries, salted hashes, role tests.	Student
R6	System is too slow	2	3	6	Low	Unchanged. Indexes defined (Section 5.7).	Test with 800 students.	Student

ID	Risk	L	I	Exp.	Level	Change at Gate 2	Mitigation	Owner
R7	Design, code and tests do not match	2	4	8	Medium	Lowered: Section 6 traces every FR to classes, tables and figures.	Extend the matrix to source files and tests at each gate.	Student
R8	Tool or library problems	3	3	9	Medium	Unchanged.	Set up CMake and CI early; keep a fallback library.	Student
R9	Rules are unclear	2	3	6	Low	Unchanged. Rules are now also expressed as Table 4.1 and Section 3.10.	Keep rules in the SRS only; change by control.	Student
R10	Stakeholders are unavailable	3	3	9	Medium	Unchanged.	Supervisor answers open questions.	Student, Supervisor
R11	Stakeholders reject the conflict resolutions	3	4	12	Medium	Unchanged. The design follows the Section 3 resolutions of Phase 2 exactly; a rejection would affect Table 4.1.	Record the Gate 1 outcome; use change control.	Student, Supervisor
R12	Argon2id makes login too slow on old computers	3	3	9	Medium	Unchanged. Parameters are held in one class, so tuning needs no other change.	Time NFR-01 on the oldest available machine.	Student
R13	Errors in deadline and penalty logic	2	4	8	Medium	Lowered: penalty isolated in IPenaltyStrategy; UTC storage fixed; boundary values designed (Table 3.5).	Boundary tests; one penalty class.	Student
R14	File handling failures	3	2	6	Low	Unchanged. IFileStore and FileException isolate file errors.	Validate files; failure tests (NFR-08).	Student
R15	NEW: Design is too large to implement in the hours available	3	4	12	Medium	Added at Gate 2.	Build in vertical slices (login, submit, mark, moderate, release); keep services thin; use the cut list.	Student

ID	Risk	L	I	Exp.	Level	Change at Gate 2	Mitigation	Owner
R16	NEW: Schema changes after Gate 2 force changes in repositories, services and tests	2	4	8	Medium	Added at Gate 2.	One versioned schema script; repositories hide tables; change control.	Student
R17	NEW: A mark change is saved without its audit record (NFR-09)	2	5	10	Medium	Added at Gate 2.	One Transaction object per operation; fault-injection tests; append-only triggers.	Student